

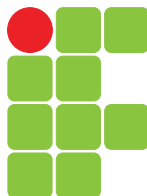
Introdução ao Python

notas de aula



Instituto Federal de Ciência e Tecnologia de Santa Catarina
Departamento Acadêmico de Eletrotécnica
Mestrado Profissional em Sistemas de Energia
Programação Aplicada a Sistemas de Energia — NVP68300

Prof. Jackson Lago, Dr.
jackson.lago@ifsc.edu.br



Sumário

Orientações Iniciais e Convenções	4
1 Variáveis e Tipos	5
1.1 Tipos primitivos	5
1.2 Collections	9
1.2.1 <code>list</code>	9
1.2.2 <code>tuple</code>	11
1.2.3 <code>set</code>	12
1.2.4 <code>dict</code>	13
1.3 Referência e mutabilidade na atribuição de variáveis	14
2 Expressões e Funções	17
2.1 Funções	17
2.2 Mutabilidade dos argumentos de funções	21
2.3 Recursão	22
3 Estruturas de controle de fluxo	24
3.1 Estruturas condicionais	25
3.1.1 <code>if-else</code>	25
3.1.2 <code>if-elif-else</code>	25
3.1.3 <code>match-case</code>	26
3.2 Estruturas de repetição	26
3.2.1 <code>while</code>	27
3.2.2 <code>for</code>	28
4 Exercícios	30
Questão 4.1: <code>bussola</code>	30
Questão 4.2: <code>desvio_padrao</code>	31
Questão 4.3: <code>login</code>	31
Questão 4.4: <code>separa_pares_impares</code>	32
Questão 4.5: <code>seno</code>	32
Questão 4.6: <code>produto_mais_vendido</code>	33
Questão 4.7: <code>alunos_aprovados</code>	33
Questão 4.8: <code>media_movel</code>	33
Questão 4.9: <code>ordena</code>	34
Questão 4.10: <code>procura</code>	35
Questão 4.11: <code>balanceamento</code>	35
Questão 4.12: <code>tictactoe</code>	36



Questão 4.13: <code>tictactoe</code> (continuação)	38
5 Classes	39
6 NumPy, Matplotlib e Pandas	44
6.1 NumPy	44
6.2 Matplotlib	46
6.3 Pandas	48
7 Exercícios	52
Questão 7.1: <code>pvcell</code>	52
Questão 7.2: <code>pll</code>	55
Questão 7.3: <code>consumo_horario</code>	58
Questão 7.4: <code>fft</code>	59
Questão 7.5: <code>inverter</code>	60

Orientações Iniciais e Convenções

Este tutorial oferece uma introdução concisa ao Python, voltada para os alunos ingressantes no curso de Mestrado Profissional em Sistemas de Energia.

Por ter um caráter introdutório, o texto apresenta exemplos e conceitos sem a intenção de esgotar completamente o tema ou fornecer soluções otimizadas para problemas específicos.

Os exemplos aqui descritos assumem a utilização de Python 3.10 (ou superior) que deve estar devidamente instalado (<https://www.python.org/downloads/>).

Também recomenda-se a utilização de alguma IDE (*Integrated Development Environment*) para facilitar a escrita e manipulação do código. As duas mais populares para desenvolvimento em Python são:

- **PyCharm:** uma IDE dedicada ao Python, desenvolvida pela JetBrains. Oferece uma configuração padrão robusta e inclui funcionalidades como autocomplete, análise de código, refatoração, debugging, testes integrados e integração com Git. É uma ferramenta profissional ideal para projetos complexos, embora possa ser pesada em máquinas mais simples ou antigas. Disponível em duas versões: Professional (paga) e Community Edition (gratuita). Embora a versão gratuita tenha algumas limitações, ela é mais do que suficiente para este tutorial e até mesmo para projetos pessoais de grande porte. (<https://www.jetbrains.com/pycharm/download/>)
- **VS Code:** um editor de código aberto, leve e flexível, não restrito a uma única linguagem. O suporte ao Python é obtido por meio de extensões, que também habilitam ferramentas de análise de código. Seu uso exige uma configuração inicial, incluindo a instalação da extensão Python (Microsoft). Outras extensões relacionadas à produtividade podem ser obtidas pelo marketplace integrado ao VS Code. (<https://code.visualstudio.com/download/>)

No texto, códigos Python serão identificados por uma caixa preta:

```
exemplo_codigo_python.py
print('Hello, World!')
```

enquanto comandos e mensagens impressas no terminal estarão em uma caixa azul:

```
C:\curso_python> python exemplo_codigo_python.py
Hello, World!
```

Capítulo 1

Variáveis e Tipos

A computação em seu nível mais básico baseia-se na manipulação de dados armazenados na memória.

As linguagens naturais são ambíguas e, muitas vezes, subjetivas, o que pode gerar interpretações variadas. Diferentemente delas, uma linguagem de programação deve ser precisa e determinística, garantindo previsibilidade no processamento da informação.

Através dessas linguagens, comunicamos ao hardware como manipular os dados armazenados na memória, transformando-os em novos valores. Esses valores, por sua vez, são interpretados e associados a algo significativo no mundo real, seja um número, um texto ou uma representação visual.

Blocos de dados armazenados na memória que representam coletivamente um conceito comum são chamados objetos. Os tipos associados a esses objetos formam uma camada de abstração que define como um conjunto de bits deve ser interpretado. Essa abstração permite representar diversos conceitos, como números inteiros, valores fracionários, caracteres e outros tipos de dados, tornando a manipulação dessas informações mais estruturada e intuitiva.

Diferentemente de algumas linguagens, em Python não é necessário declarar explicitamente o tipo de uma variável, pois o interpretador determina automaticamente esse tipo com base no valor atribuído.

Python possui diversos tipos primitivos, que representam os dados mais básicos da linguagem. Estes podem ser utilizados diretamente para expressar algum conceito real ou combinados para formar tipos compostos (ou estruturados), capazes de representar ideias e relações mais complexas.

1.1: Tipos primitivos

Os principais tipos primitivos em Python são:

- `int`: utilizado para armazenar números inteiros, como `10`, `-5`, e `0`
- `float`: representa valores com casas decimais, como `3.14`, `-0.5`, `1.6e-3`
- `complex`: representa números complexos, como `3.2 + 4.7j`



- **str**: string utilizado para armazenar textos, como `'Hello, world!'` ou `"python"`¹.
- **bool**: representa uma grandeza booleana, que pode assumir apenas `True` ou `False`, essencial para estruturas de decisão
- **NoneType**: pode assumir apenas o valor `None`, e pode ser explicitamente atribuído a uma variável para sinalizar a ausência de um valor.

Em código a criação de variáveis com os tipos acima descritos ficaria:

```
idade = 25
peso = 78.6
impedancia = 5 + 3j
ativo = True
nome = 'Fulano de Tal'
sem_valor = None

print(f"'idade' é uma variável do tipo {type(idade)} e valor: {idade}")
print(f"'peso' é uma variável do tipo {type(peso)} e valor: {peso}")
print(f"'impedancia' é uma variável do tipo {type(impedancia)} e valor: {impedancia}")
print(f"'ativo' é uma variável do tipo {type(ativo)} e valor: {ativo}")
print(f"'nome' é uma variável do tipo {type(nome)} e valor: {nome}")
print(f"'sem_valor' é uma variável do tipo {type(sem_valor)} e valor: {sem_valor}")
```

O código acima apenas cria diversas variáveis de diferentes tipos e para cada uma delas imprime o seu tipo e valor no terminal:

```
'idade' é uma variável do tipo <class 'int'> e valor: 25
'peso' é uma variável do tipo <class 'float'> e valor: 78.6
'impedancia' é uma variável do tipo <class 'complex'> e valor: (5+3j)
'ativo' é uma variável do tipo <class 'bool'> e valor: True
'nome' é uma variável do tipo <class 'str'> e valor: Fulano de Tal
'sem_valor' é uma variável do tipo <class 'NoneType'> e valor: None
```

Antes de prosseguirmos e começarmos a utilizar essas variáveis para realizar tarefas úteis, é importante refletir sobre o conceito de tipagem e as diferentes formas como as linguagens de programação a tratam.

Algumas linguagens possuem tipagem dinâmica, onde o tipo das variáveis é determinado durante a execução do programa. Outras adotam tipagem estática, exigindo que os tipos sejam declarados antecipadamente e validados durante a compilação (quando aplicável). Essa diferença impacta diretamente a maneira como escrevemos, mantemos e depuramos código.

Python é uma linguagem interpretada e de tipagem dinâmica, o que significa que seu código não é compilado previamente e que o tipo de uma variável é determinado em tempo de execução, sem exigir uma declaração explícita por parte do programador. Isso não significa que as variáveis não possuam um tipo associado, mas sim que o próprio interpretador do Python gerencia automaticamente os tipos conforme necessário.

Por exemplo, no seguinte código:

```
descricao = 'Geladeira frost free 500L' # tipo: str
quantidade = 10 # tipo: int
peso_unitario = 72.7 # tipo: float
```

as variáveis `descricao`, `quantidade` e `peso_unitario`² são automaticamente criadas com os tipos `str`, `int` e `float`, respectivamente. O interpretador infere esses tipos com base nos valores atribuídos a cada variável (o valor à direita de `=`).

¹Em Python, strings literais podem ser delimitadas por aspas simples (`'`) ou duplas (`"`). Essa flexibilidade é útil quando queremos incluir o outro tipo de aspas dentro do texto, sem a necessidade de caracteres de escape.

²Em Python, o estilo `snake_case` é tradicionalmente usado para nomear variáveis, separando palavras com underscores (`_`) e usando minúsculas.



Além disso, uma variável pode ser redefinida posteriormente, até mesmo para um tipo diferente. Essa abordagem pode ser vantajosa para tornar o código mais flexível e dinâmico, mas também pode introduzir problemas ao mascarar erros difíceis de identificar.

Por exemplo, em

```
descricao = 'Geladeira frost free 500l' # tipo: str
quantidade = 10 # tipo: int
peso_unitario = 72.7 # tipo: float

print(f"pré: 'descricao' é do tipo {type(descricao)} com valor: {descricao}")
descricao = quantidade # redefine descricao para int
print(f"pós: 'descricao' é do tipo {type(descricao)} com valor: {descricao}")
```

```
pré: 'descricao' é do tipo <class 'str'> com valor: Geladeira frost free 500l
pós: 'descricao' é do tipo <class 'int'> com valor: 10
```

a variável `descricao` inicialmente criada como `str` e valor `'Geladeira frost free 500l'`, é redefinida para `int`, passando a armazenar o valor `10`. Esse comportamento exemplifica a tipagem dinâmica do Python, onde uma mesma variável pode assumir diferentes tipos ao longo da execução do código.

Em Python, uma variável é apenas um nome que referencia um objeto em memória. Quando uma variável é redefinida, ela passa a apontar para outro objeto, que não necessariamente precisa ter o mesmo tipo.

Esse código é válido e não resultará em erros de execução por si só. No entanto, pode representar um erro lógico caso essa alteração de tipo não esteja alinhada com a intenção do programador. Cabe ao programador garantir a coerência do código, tratando `descricao` como `str` na parte inicial do programa e como `int` posteriormente, conforme necessário.

Embora essa redefinição com mudança de tipo não configure um erro sintático e possa ser útil em alguns casos, sua prática indiscriminada é desencorajada, pois pode dificultar legibilidade e a manutenção do código.

A tipagem dinâmica do Python torna a linguagem mais simples e flexível, facilitando o desenvolvimento, especialmente para iniciantes. No entanto, essa característica exige atenção, pois erros de tipagem podem surgir em tempo de execução se os tipos das variáveis não forem corretamente tratados.

Por exemplo, considere o seguinte código:

```
descricao = 'Geladeira frost free 500l' # tipo: str
quantidade = 10 # tipo: int
peso_unitario = 72.7 # tipo: float

peso_total = peso_unitario * quantidade
print(f'Peso total: {peso_total} kg')
```

```
Peso total: 727.0 kg
```

As variáveis `quantidade` (`int`) e `peso_unitario` (`float`) são multiplicadas, gerando um novo valor do tipo `float`, que é atribuído a `peso_total`. Essa operação é perfeitamente válida, tanto do ponto de vista sintático da linguagem quanto do ponto de vista lógico.

Agora, observe o que aconteceria se o programador, por descuido, digitasse `descricao` ao invés de `peso_unitario`.



```
descricao = 'Geladeira frost free 500l' # tipo: str
quantidade = 10 # tipo: int
peso_unitario = 72.7 # tipo: float

peso_total = peso_unitario * descricao # <- erro de digitação
print(f'Peso total: {peso_total} kg')
```

```
Traceback (most recent call last):
  File "c:\curso_python\aula1\variaveis.py", line 5, in <module>
    peso_total = peso_unitario * descricao
                  ~~~~~^~~~~~
TypeError: can't multiply sequence by non-int of type 'float'
```

Durante a execução do código, o interpretador encontrou uma operação inválida e gerou um erro de incompatibilidade de tipos, pois a multiplicação entre um `float` com um `str` não está definida.

Perceba que esse erro só ocorre em tempo de execução, no momento em que o interpretador Python tenta realizar a multiplicação entre um `float` e um `str`. Trata-se de um erro de lógica, não de sintaxe, pois, sendo Python uma linguagem de tipagem dinâmica, `peso_unitario` e `descricao` poderiam armazenar qualquer tipo de dado ao longo da execução do código, podendo sofrer mudanças não só de valor, mas também de tipo, até atingir a linha onde ocorre a tentativa de multiplicação e, consequentemente, o erro.

Esse erro seria facilmente detectado e corrigido no momento da escrita do código ou durante a compilação em linguagens de tipagem estática (como C, C++, Java, Rust, etc), impedindo sua ocorrência em tempo de execução. Esse exemplo foi incluído apenas para ilustrar também as desvantagens e os possíveis desafios da tipagem dinâmica.

Embora Python não suporte tipagem estática, ele suporta opcionalmente anotação de tipos. As anotações de tipos são declarações do programador quanto ao tipo pretendido para cada variável criada³.

Apesar de as anotações de tipos não influenciarem a execução do código, sendo ignoradas pelo interpretador Python durante a execução, elas são extremamente úteis. Essas anotações são utilizadas por ferramentas de análise estática integradas à maioria das IDEs modernas para identificar possíveis erros de incompatibilidade de tipos no momento da escrita do código⁴ além de melhorar a legibilidade e a manutenção, minimizando assim os riscos associados à tipagem dinâmica, mantendo todos os seus benefícios.

Reescrevendo o exemplo anterior com anotação de tipos, a IDE agora pode nos alertar sobre um provável erro antes mesmo da execução:

```
descricao: str = 'Geladeira frost free 500l'
quantidade: int = 10
peso_unitario: float = 72.7

peso_total = peso_unitario * descricao # <- erro de digitação
print(f'Peso total: {peso_total} kg')
```

Operator '*' not supported for types 'float' and 'str'

Além dos tipos primitivos, Python possui mais duas formas de armazenar informações: através de classes customizadas `class`, que serão discutidas no capítulo 5 e através de coleções de dados (`collections`).

³Além da anotação de tipo de variáveis, podemos também anotar tipos para parâmetros e retorno de funções/métodos

⁴A análise estática de tipos (ou *type checking*) está habilitada por padrão na IDE PyCham. Já no VS Code, ela pode ser habilitada incluindo os campos `"python.analysis.autoImportCompletions": true`, `"python.analysis.typeCheckingMode": "standard"` no arquivo de configurações `settings.json`, encontrado com `F1 → Preferences: Open User Settings (JSON)`.



1.2: Collections

Em Python, coleções são estruturas que armazenam múltiplos valores em uma única variável. As quatro coleções mais comuns da linguagem são: `list`, `tuple`, `set` e `dict`. Cada uma delas possui características específicas que as tornam mais adequadas para diferentes situações.

1.2.1: `list`

As listas (`list`) são uma das estruturas mais versáteis do Python, representam sequências de elementos ordenados e mutáveis. Ou seja, os elementos inseridos nela são mantidos na mesma ordem e podem ser alterados a qualquer momento, seja adicionando, removendo modificando ou substituindo itens.

Além disso, possuem tamanho arbitrário, crescendo conforme novos elementos são inseridos.

Uma lista pode ser criada utilizando colchetes (`[]`) e pode armazenar qualquer tipo de dado, inclusive, diferentes tipos para cada elemento. Por exemplo:

```
numeros = [8, 2, 0, -4, 15]          # lista de inteiros
nomes = ['Fulano', 'Beltrano', 'Sicrano'] # lista de strings
misto = [42, 'texto', 3.14, True]     # lista com tipos variados
```

A lista é uma coleção dinâmica, e pode crescer mesmo após sua criação para acomodar novos elementos:

```
cores = ['azul', 'amarelo', 'cinza', 'vermelho']
print(f"pré: A lista 'cores' possui {len(cores)} elementos: {cores}")

cores.append('verde') # adiciona um novo elemento de valor 'verde' ao final da lista
print(f"pós: A lista 'cores' possui {len(cores)} elementos: {cores}")

pré: A lista 'cores' possui 4 elementos: ['azul', 'amarelo', 'cinza', 'vermelho']
pós: A lista 'cores' possui 5 elementos: ['azul', 'amarelo', 'cinza', 'vermelho', 'verde']
```

Esse código introduz um elemento novo: a chamada `cores.append()`, onde `append()` é um método da classe `list`. Métodos são semelhantes a funções, mas estão associados a um tipo de dado específico (a uma `class`) e normalmente modificam o próprio objeto ao qual pertencem.

A chamada de um método geralmente impacta diretamente o objeto a partir do qual ele é invocado. No caso de `append()`, um novo elemento é adicionado ao final da lista, alterando sua estrutura inicial.

Esse conceito será revisitado e formalizado com mais detalhes no Capítulo 5, quando exploraremos a construção e uso de classes em Python.

Note que a lista cresce automaticamente para acomodar novos elementos, sem que o programador precise gerenciar ou alocar memória adicional manualmente. Essa característica torna as listas particularmente flexíveis para armazenar conjuntos dinâmicos de dados.

Além disso, no último exemplo, introduzimos a função `len()`, que retorna o número de elementos presentes na coleção. Essa é uma função essencial para validar o tamanho de uma lista antes de realizar operações que dependam da quantidade de itens.

Também podemos acessar elementos individuais dessa lista, tanto para leitura como para escrita. Essa indexação é realizada com a seguinte sintaxe `nome_da_lista[indice]`. Em python a indexação inicia em `0`. Veja o código exemplo:



```
cores = ['azul', 'amarelo', 'cinza', 'vermelho', 'verde']  
  
selecionada1 = cores[1]    # índice 1 equivale ao 2º elemento  
print(f'selecioanda1 é: {selecionada1}')
```

```
seleccioanda1 é: amarelo
```

Índices negativos podem ser usados para indexar elementos do final para o início da listas:

```
selecionada2 = cores[-2]   # -2 indica o penúltimo elemento  
print(f'seleccioanda2 é: {selecionada2}')
```

```
seleccioanda2 é: vermelho
```

É possível a indexação por `slice`, ou seja, po faixa de índices `m:n` (`n` é não inclusivo):

```
selecionadas = cores[0:3]  # 0:3 corresponde aos índices 0,1,2  
print(f'selecionadas são: {selecionadas}')
```

```
selecionadas são: ['azul', 'amarelo', 'cinza']
```

Elementos da lista também podem ser individualmente modificados:

```
cores[2] = 'marrom'        # altera o terceiro elemento de 'cinza' para 'marrom'  
print(f'pós modificação: {cores}')
```

```
pós modificação: ['azul', 'amarelo', 'marrom', 'vermelho', 'verde']
```

Podemo excluir elementos, o que desloca os índices dos elementos a sua direita:

```
del cores[3]               # deleta o 4º elemento  
print(f'pós excusão por índice: {cores}')
```

```
pós excusão por índice: ['azul', 'amarelo', 'marrom', 'verde']
```

Podemos ainda excluir não por índice, mas por valor:

```
cores.remove('azul')  
print(f'pós excusão por valor: {cores}')
```

```
pós excusão por valor: ['amarelo', 'marrom', 'verde']
```

Tentar acessar índices que não existe resultará em um erro:

```
cores[99]
```

```
Traceback (most recent call last):  
  File "c:\curso_python\aula1\variaveis.py", line 28, in <module>  
    cores[99]  
    ~~~~~^  
IndexError: list index out of range
```

Vários outros métodos e funções para manipular listas estão disponíveis por padrão. Segue uma breve enumeração dos mais úteis:

- `len(lista)` – Retorna o número de elementos na lista.
- `lista.append(valor)` – Adiciona um novo elemento ao final da lista.
- `lista.insert(indice, valor)` – Insere um elemento em uma posição específica.
- `lista.remove(valor)` – Remove a primeira ocorrência do valor especificado.
- `lista.pop(indice)` – Remove e retorna o elemento na posição indicada (ou o último, se omitido).



- `lista.index(valor)` – Retorna o índice da primeira ocorrência do valor.
- `lista.count(valor)` – Retorna o número de vezes que o valor aparece na lista.
- `lista.sort()` – Ordena a lista em ordem crescente (por padrão).
- `lista.reverse()` – Inverte a ordem dos elementos da lista.
- `sorted(lista)` – Retorna uma nova lista ordenada, sem modificar a original.
- `lista.copy()` – Retorna uma cópia superficial da lista.
- `lista.clear()` – Remove todos os elementos da lista.
- `lista[i:j:k]` – fatiamento (*slicing*) de `i` até `j` com passo `k`.

Esses métodos permitem desde operações simples, como acessar elementos, até manipulações mais avançadas, como ordenação e remoção de elementos. No decorrer desse tutorial, abordaremos mais detalhes sobre a aplicação desses métodos em exemplos práticos.

1.2.2: tuple

Assim como `list`, a `tuple` (ou tupla) é uma estrutura de dados que permite armazenar uma sequência de valores. No entanto, enquanto listas são mutáveis (ou seja, seus elementos podem ser alterados, adicionados ou removidos), tuplas são imutáveis: uma vez criada, seus elementos não podem ser modificados.

Essa característica confere à tupla maior segurança e previsibilidade, tornando-a ideal para representar coleções fixas de dados que não devem ser alteradas acidentalmente, ou até valores de retorno de uma função.

Tuplas são definidas por uma sequência de valores separados por vírgula (`,`), não necessariamente, mas comumente inseridos entre parênteses (`()`).

```
coordenada = (46.8, 22.3)
print(coordenada)
```

```
(46.8, 22.3)
```

Para definir uma tupla de um único elemento, é obrigatório adicionar uma vírgula, caso contrário, o Python interpretará o valor como um tipo isolado:

```
numero = (5)    # não é tupla, é apenas um int
tupla = (5,)    # tupla com um único elemento
print(f'numero é do tipo {type(numero)} e valor: {numero}')
print(f'tupla é do tipo {type(tupla)} e valor: {tupla}')
```

```
numero é do tipo <class 'int'> e valor: 5
tupla é do tipo <class 'tuple'> e valor: (5,)
```

A indexação de tuplas segue o mesmo princípio das listas, permitindo acessar elementos por índices (0 para o primeiro item, índices negativos para contar do final)

```
cores = 'azul', 'verde', 'vermelho', 'amarelo', 'roxo'
print(cores[0])    # azul
print(cores[-1])   # roxo
print(cores[1:4])  # ('verde', 'vermelho', 'amarelo')
```

```
azul
roxo
('verde', 'vermelho', 'amarelo')
```



Embora tupla possa parecer uma versão limitada de lista, elas são estruturas complementares, com propósitos distintos. Essas limitações justamente tornam operações com tuplas mais previsíveis, rápidas e seguras.

1.2.3: `set`

Um `set` é uma coleção semelhante à lista (`list`), porém não permite valores duplicados e não mantém uma ordem fixa dos elementos. Isso faz com que seja uma estrutura útil para armazenar itens únicos e realizar operações como união, interseção e diferença.

Os conjuntos podem ser criados utilizando chaves `{}` ou a função `set()`:

```
cores = {'azul', 'vermelho', 'verde', 'azul'}  
print(cores)
```

```
{'azul', 'vermelho', 'verde'}
```

Observe que, mesmo que `'azul'` tenha sido declarado duas vezes, ele aparece apenas uma vez no conjunto.

Outra forma de criar um conjunto é utilizando `set()` passando uma `list` como argumento:

```
valores = set([1, 2, 3, 4, 4, 5])  
print(valores)
```

```
{1, 2, 3, 4, 5}
```

Os conjuntos são ideais quando precisamos garantir que os elementos sejam únicos, eliminando valores repetidos automaticamente.

Por não manter uma ordem fixa dos elementos, `set` não permite acesso por índice. Isso significa que não podemos utilizar a notação `set[indice]` para recuperar ou modificar elementos específicos, como fazemos com listas. Em vez disso, a manipulação de conjuntos é realizada por métodos próprios, como adição, remoção, união, interseção e diferença, que operam sobre o conjunto como um todo:

```
a = {1, 2, 3, 4}  
b = {3, 4, 5}  
b.add(6)  
  
print(f'união: {a | b}')  
print(f'interseção: {a & b}')  
print(f'diferença: {a - b}')
```

```
união: {1, 2, 3, 4, 5, 6}  
interseção: {3, 4}  
diferença: {1, 2}
```

Embora `set` não suporte indexação direta, ainda é possível iterar sobre seus elementos utilizando laços como `for`. Dessa forma, podemos percorrer todos os itens do conjunto sem precisar acessar um elemento específico por índice. A iteração sobre coleções de dados será abordada no capítulo 3.

Aqui está uma lista resumida dos principais métodos e funções para manipulação de `set` em Python:

- `len(set)` – Retorna o número de elementos no conjunto.
- `set.add(valor)` – Adiciona um novo elemento ao conjunto.
- `set.remove(valor)` – Remove um elemento existente (gera erro se o valor não existir).



- `set.discard(valor)` – Remove um elemento sem gerar erro caso ele não exista.
- `set.pop()` – Remove e retorna um elemento aleatório do conjunto.
- `set.clear()` – Remove todos os elementos do conjunto.
- `set.copy()` – Retorna uma cópia do conjunto.

1.2.4: dict

Em Python, um dicionário (`dict`) é uma estrutura de dados que implementa uma *hash table*, permitindo o armazenamento de pares `key: value` (chave e valor). Essa abordagem garante acesso eficiente aos valores a partir de suas chaves, tornando a busca em grandes coleções extremamente rápida.

Diferente das listas (`list`) e conjuntos (`set`), que organizam elementos sequencialmente ou de forma desordenada, o dicionário associa cada valor a uma chave única, permitindo acessos diretos sem necessidade de percorrer toda a estrutura. Na prática, essas chaves funcionam como os índices em listas.

Cada chave (`key`) pode ser qualquer tipo de dado que suporte a função `hash()`, como números, strings ou tuplas imutáveis. Já o valor (`value`) pode ser qualquer objeto do Python, incluindo listas, outras coleções, classes definidas pelo usuário e até mesmo funções.

Podemos criar um dicionário utilizando chaves (`{}`) e inserindo os pares `key: value` dentro delas. Por exemplo:

```
dados = {'nome': 'Fulano', 'idade': 25, 'cidade': 'Florianópolis'}  
print(dados)
```

```
{'nome': 'Fulano', 'idade': 25, 'cidade': 'Florianópolis'}
```

Para recuperar um valor armazenado em um dicionário, utilizamos sua chave entre colchetes `[]`:

```
dados = {'nome': 'Fulano', 'idade': 25, 'cidade': 'Florianópolis'}  
nome = dados['nome']  
print(f'nome: {nome}')
```

```
nome: Fulano
```

Tentar acessar uma chave inexistente gera um erro `KeyError`:

```
dados = {'nome': 'Fulano', 'idade': 25, 'cidade': 'Florianópolis'}  
telefone = dados['telefone']
```

```
Traceback (most recent call last):  
  File "c:\curso_python\aula1\variaveis.py", line 2, in <module>  
    telefone = dados['telefone']  
~~~~~  
KeyError: 'telefone'
```

Já escrever em uma chave previamente inexistente, expande o `dict` incluindo esse novo par:

```
dados = {'nome': 'Fulano', 'idade': 25, 'cidade': 'Florianópolis'}  
  
dados['telefone'] = '(48)99999-9999'  
print(dados)
```

```
{'nome': 'Fulano', 'idade': 25, 'cidade': 'Florianópolis', 'telefone': '(48)99999-9999'}
```

Dicionários são mutáveis, permitindo que seus elementos sejam alterados, adicionados ou removidos. Podemos modificar ou remover um valor acessando sua chave diretamente:



```
dados = {'nome': 'Fulano', 'idade': 25, 'cidade': 'Florianópolis'}  
  
dados['idade'] = 26 # modifica o valor associado à chave 'idade'  
del dados['cidade'] # remove o valor associado à chave 'cidade'  
print(dados)
```

```
{'nome': 'Fulano', 'idade': 26}
```

Aqui estão alguns métodos úteis para trabalhar com dicionários:

- `len(dicionario)` – Retorna a quantidade de pares chave-valor no dicionário.
- `dicionario.keys()` – Retorna todas as chaves do dicionário.
- `dicionario.values()` – Retorna todos os valores do dicionário.
- `dicionario.items()` – Retorna todos os pares chave-valor como tuplas.
- `dicionario.pop(chave)` – Remove um item e retorna seu valor.
- `dicionario.update(outro_dicionario)` – Atualiza o dicionário com novos pares chave-valor.
- `dicionario.clear()` – Remove todos os elementos do dicionário.

Dicionários são ferramentas poderosas para armazenar e acessar dados de forma eficiente. Eles são amplamente utilizados em Python para estruturar informações e otimizar operações de busca.

1.3: Referência e mutabilidade na atribuição de variáveis

Em Python, as variáveis não armazenam diretamente os valores, mas referências para objetos na memória. Dessa forma, o efeito da atribuição (`=`) a uma variável depende da mutabilidade do objeto que ela referencia, definindo se a variável continuará apontando para o mesmo objeto ou passará a referenciar um novo.

Os objetos em Python são divididos em dois grupos: imutáveis e mutáveis, conforme sua capacidade de sofrer alterações após a criação.

Objetos imutáveis, como `int`, `float`, `str`, `bool` e `tuple`, não podem ser modificados após sua criação. Qualquer operação que tente alterar uma variável que referencia um imutável resulta na criação de um novo objeto, com um novo endereço de memória, e a variável passa a apontar para ele, sem afetar o objeto original.

Por outro lado, objetos mutáveis, como `list`, `dict`, `set` e instâncias de classes, permitem alterações diretas em seu conteúdo sem a necessidade de criar um novo objeto. Assim, modificações feitas em uma referência são refletidas em todas as variáveis que apontam para o mesmo objeto.

Veja um exemplo com variáveis imutáveis, como `int`:

```
x = 10 # x referencia um inteiro imutável  
print(f"{'type(x)=}', {'id(x)=}', {'x'})")  
  
y = x # y agora referencia o mesmo objeto que x  
print(f"{'type(y)=}', {'id(y)=}', {'y'})")  
  
y = y + 5 # um novo objeto é criado e y passa a referenciá-lo  
print(f"{'type(y)=}', {'id(y)=}', {'y'})")
```



```
z = 10 # uma nova variável z referencia o inteiro 10
print(f"{'type(z)=', {id(z)=}, {z}")
```

```
type(x)=<class 'int'>, id(x)=140706534401224, 10
type(y)=<class 'int'>, id(y)=140706534401224, 10
type(y)=<class 'int'>, id(y)=140706534401384, 15
type(z)=<class 'int'>, id(z)=140706534401224, 10
```

A função `id(x)`, utilizada em `print`, retorna o identificador único do objeto referenciado por `x`, que equivale ao seu endereço de memória na implementação CPython.

A saída do código mostra que `x` e `y` inicialmente referenciam o mesmo objeto `10`, compartilhando o mesmo identificador único (`id`). No entanto, ao modificar `y`, Python cria um novo objeto com valor `15`, e `y` passa a apontar para ele, sem alterar `x`.

Já `z`, criado posteriormente, também referencia o objeto `10`. Como esse valor é um inteiro imutável, Python pode reutilizar a mesma referência na memória⁵, garantindo que o objeto permaneça inalterável. Dessa forma, qualquer modificação em `x` faz com que ele passe a apontar para outro objeto, sem impactar `z`.

Agora, um exemplo similar com variáveis mutáveis (lista de um elemento inteiro):

```
x = [10] # x referencia um lista mutável com apenas um elemento
print(f"{'type(x)=', {id(x)=}, {x}")

y = x # y agora referencia o mesmo objeto que x
print(f"{'type(y)=', {id(y)=}, {y}")

y[0] = y[0] + 5 # nenhum novo objeto é criado, o valor em y[0] é alterado
print(f"{'type(y)=', {id(y)=}, {y}")

z = [10] # uma nova variável z, idêntica, mas independente de x
print(f"{'type(z)=', {id(z)=}, {z}")
```

```
type(x)=<class 'list'>, id(x)=1945482649600, [10]
type(y)=<class 'list'>, id(y)=1945482649600, [10]
type(y)=<class 'list'>, id(y)=1945482649600, [15]
type(z)=<class 'list'>, id(z)=1945484459648, [10]
```

Neste caso, `x` e `y` são referências para a mesma lista, que é mutável. Ao modificar `y[0]`, o conteúdo da lista é alterado diretamente, sem criar um novo objeto, o que é evidenciado pelo fato de que o identificador único (`id`) permanece o mesmo.

Já `z` contém outra lista `[10]`, alocada em um novo espaço de memória, o que explica seu identificador diferente.

Caso se deseje copiar os valores de uma lista para outra, em vez de apenas referenciar o mesmo objeto, é necessário usar `copy()` ou `deepcopy()`.

A função `copy()` cria uma nova lista contendo os mesmos elementos da lista original, mas se algum elemento for um objeto mutável, como outra lista ou um dicionário, a cópia conterá apenas referências para esses objetos, e modificações nos elementos internos afetarão ambas as listas.

Já `deepcopy()` copia recursivamente todos os objetos mutáveis dentro da estrutura, garantindo que cada nível da cópia seja independente da original.

Veja o exemplo:

⁵Essa reutilização de objetos é uma otimização da implementação CPython conhecida como *internin*, geralmente aplicada para pequenos objetos.



```
from copy import deepcopy

x = [10]    # lista mutável com apenas um elemento
print(f"type(x)={}, {id(x)}={}, {x}")

y = x      # y agora referencia o mesmo objeto que x
print(f"type(y)={}, {id(y)}={}, {y}")

z = deepcopy(x)    # nova lista com cópias dos valores de x
print(f"type(z)={}, {id(z)}={}, {z}")
```

```
type(x)=<class 'list'>, id(x)=2511057429440, [10]
type(y)=<class 'list'>, id(y)=2511057429440, [10]
type(z)=<class 'list'>, id(z)=2511059612160, [10]
```

A compreensão da mutabilidade será crucial quando analisarmos a passagem de argumentos para funções, tema abordado no próximo capítulo.

Capítulo 2

Expressões e Funções

Uma expressão é qualquer fragmento de código que, ao ser executado, retorna um valor. Ela pode ser composta por valores, variáveis, operadores e chamadas de funções, que são avaliadas para produzir um resultado. Esse resultado pode ser armazenado em uma variável ou utilizado como entrada para outra expressão ou função.

Um exemplo comum de expressão são as operações aritméticas:

```
a = 3
b = 7

y = a * 5 + b
```

Aqui, as variáveis `a` e `b` são inicializadas com valores fixos (ou *literals*, como são chamados em programação), enquanto `y` recebe o resultado do processamento da expressão `a * 5 + b`, que retorna 22.

Os operadores aritméticos em Python são: adição (+), subtração (-), multiplicação (*), divisão (/), divisão inteira (//), resto da divisão (%) e potenciação (**).

2.1: Funções

Outro pilar essencial da programação são as funções, que permitem a organização e reutilização de código. Elas ajudam a evitar repetições desnecessárias, encapsular blocos lógicos e contribuir para uma estrutura mais eficiente e modular.

Uma função é um bloco de código reutilizável, projetado para executar uma ação específica. Sua declaração possui três elementos fundamentais:

1. Parâmetros de entrada, que atuam como espaços reservados para valores que serão fornecidos posteriormente
2. quando a função for chamada.
3. O corpo da função, onde a lógica é definida e executada.
4. O valor de retorno, que é o resultado da função após seu processamento.

Quando chamamos a função, fornecemos argumentos, que são os valores reais atribuídos aos parâmetros de entrada. Esses argumentos carregam informações essenciais vindas do contexto



externo, ou seja, da parte do programa que chamou a função. A função, então, processa esses dados e executa a lógica necessária para produzir um resultado.

Outro aspecto crucial das funções é o escopo de suas variáveis locais (internas). Variáveis criadas dentro de uma função existem apenas durante sua execução e são automaticamente descartadas ao seu término. Isso evita interferências indesejadas em outras partes do código e melhora a eficiência do uso de memória.

Para ilustrar esse conceito, podemos criar a função `eq_reduzida_reta()`, que encapsula o processamento da expressão matemática utilizada anteriormente. Nesse caso, ela receberá três parâmetros de entrada: `a`, `b` e `x`. No corpo da função, a expressão matemática será avaliada para calcular `y`, que será então retornado como resultado.

A chamada a essa função constitui uma expressão, de forma que esse valor de retorno pode ser capturado e armazenado em uma variável ou usada como valor para outra expressão, ou ainda como argumento para outra função.

Veja como essa função pode ser definida em Python:

```
# definição da função
# não executa nenhum código, apenas define o que será executado quando a função for chamada
def eq_reduzida_reta(a, x, b):
    y = a * x + b
    return y

# chamada da função com diferentes argumentos
resultado1 = eq_reduzida_reta(3, 5, 7) # argumentos posicionais 3, 5, 7
resultado2 = 5 + eq_reduzida_reta(4, 2, 6) # resultado como entrada para uma expressão
resultado3 = eq_reduzida_reta(x=-2, b=10, a=2) # argumentos nomeados

print(f'resultado1: {resultado1}')
print(f'resultado2: {resultado2}')
print(f'resultado3: {resultado3}')
```

```
resultado1: 22
resultado2: 19
resultado3: 6
```

A declaração da função é indicada pela *keyword* `def`, seguida do nome da função e pela lista de parâmetros de entrada, declarados entre parênteses. Após os dois pontos (`:`), inicia-se o corpo da função.

Diferente de algumas outras linguagens onde a indentação é apenas estética, em Python ela é essencial, pois define o bloco de código pertencente à função. Isso garante a organização da estrutura e evita ambiguidades na execução.

Por fim, o comando `return` especifica o valor que será retornado ao invocador da função. Se omitida, a função não retorna um valor explícito, e seu retorno será implicitamente `None`.

Uma chamada de função é feita pelo seu nome, seguido por um par de parênteses. Dentro dos parênteses, passamos os argumentos, que devem corresponder aos parâmetros definidos pela função.

Em Python, essa correspondência pode ocorrer de duas formas:

- Argumentos posicionais, em que os valores são passados na ordem definida pela função.
- Argumentos nomeados, onde especificamos explicitamente qual valor será atribuído a cada parâmetro, permitindo uma ordem arbitrária.



Também podemos combinar ambos os tipos de argumentos em uma chamada de função. No entanto, os argumentos posicionais devem obrigatoriamente vir primeiro, seguidos pelos nomeados.

Python também permite a definição de parâmetros opcionais, o que proporciona maior flexibilidade na chamada de funções. Isso significa que ao invocar uma função, podemos fornecer todos os argumentos necessários, ou omitir aqueles marcados como opcionais.

Um parâmetro é considerado opcional quando, na definição da função, atribuímos a ele um valor padrão. Esse valor será usado automaticamente caso o chamador da função não forneça um argumento correspondente.

Por exemplo, podemos reescrever a função `eq_reduzida_reta()` tornando `b` um parâmetro opcional, atribuindo a ele um valor padrão de `0`. O código correspondente seria:

```
# definição da função com parâmetro opcional
def eq_reduzida_reta(a, x, b=0):
    y = a * x + b
    return y

# chamada da função com diferentes quantidades de argumentos
resultado1 = eq_reduzida_reta(3, 5, 7) # arg b = 7 (fornecido explicitamente)
resultado2 = eq_reduzida_reta(3, 5)    # arg b omitido, valor padrão é usado (b=0)

print(f'resultado1: {resultado1}')
print(f'resultado2: {resultado2}')
```

```
resultado1: 22
resultado2: 15
```

Esse conceito permite maior flexibilidade na chamada de funções, reduzindo a necessidade de fornecer todos os argumentos manualmente quando faz sentido na lógica da função possuir comportamento padrão. Se não indicarmos um valor para `b`, a função usará automaticamente `b=0`. Os demais parâmetros (`a` e `x`) não possuem valor padrão, portanto, são obrigatórios. Chamar a função sem fornecê-los resultaria em um erro.

Assim como na criação de variáveis, também podemos anotar os tipos dos parâmetros e do valor de retorno de uma função. Essas anotações não afetam a execução do programa, pois são ignoradas pelo interpretador durante a execução. No entanto, elas podem ser analisadas por um *type checker* da IDE, que alerta sobre possíveis incompatibilidades antes da execução do código.

Isso melhora a legibilidade do código e ajuda na detecção de erros potenciais, tornando a programação mais segura e organizada, além de documentar as intenções do programador quando escreveu tal função.

```
# definição da função com parâmetro opcional e anotação de tipos esperados
def eq_reduzida_reta(a: int, x: int, b: int = 0) -> int:
    y = a * x + b
    return y

# chamada da função
resultado1 = eq_reduzida_reta(3, 5.6, 7) # 'float' incompatível com declaração
resultado2 = eq_reduzida_reta(3, 'texto') # 'str' incompatível com declaração
resultado3 = eq_reduzida_reta(3, 5, 7)    # ok
```

Aqui, todos os parâmetros de entrada e o valor de retorno foram definidos com o tipo `int`, o que garante que apenas números inteiros sejam aceitos. Se um valor `str` ou `float` for passado incorretamente, um *type checker* pode alertar sobre a incompatibilidade.



Contudo, para essa função em específico, números fracionários (`float`) seriam completamente válidos. Para permitir esse tipo de entrada, podemos utilizar anotações de múltiplos tipos com o operador `|`, como no exemplo abaixo:

```
# Definição da função com suporte a múltiplos tipo
def eq_reduzida_reta(a: int | float, x: int | float, b: int | float = 0) -> int | float:
    y = a * x + b
    return y
```

Além disso, nada impede que parâmetros e valores de retorno tenham tipos diferentes—tudo depende da lógica que a função deve executar. A anotação de tipos é opcional, mas é altamente encorajada para manter a clareza do código e detectar possíveis erros antes da execução.

Inda sobre funções, os parâmetros de entrada e o valor de retorno são elementos opcionais em uma função e sua presença depende da ação que se deseja realizar. Algumas funções podem simplesmente executar uma tarefa sem receber valores externos ou sem produzir e retornar um resultado.

Um exemplo disso é a função padrão `print`, que já utilizamos ao longo deste material sem muitas explicações. Essa função recebe um argumento, geralmente uma `str` contendo o texto a ser exibido, e imprime essa informação no terminal. No entanto, ela não retorna um valor¹ para quem a chamou no código.

Internamente ela apenas faz uma chamada para o sistema operacional para imprimir um texto no terminal.

Em Python, funções também podem retornar múltiplos valores simultaneamente, o que torna o código mais flexível e evita a necessidade de usar estruturas, listas ou dicionários para armazenar múltiplos resultados.

Essa funcionalidade é especialmente útil quando uma função precisa retornar vários cálculos ou informações relacionadas ao mesmo tempo, além de possibilitar o retorno de dados sobre erros encontrados durante sua execução

Isso é possível porque Python permite que uma função retorne uma `tupla`, que pode ser desempacotada na chamada da função. Veja um exemplo:

```
import math

def bhaskara(a: float, b: float, c: float) -> tuple[float, float]:
    delta = b**2 - 4*a*c
    if delta < 0:
        raise ValueError("A equação não possui raízes reais. O delta é negativo.")

    raiz1 = (-b + math.sqrt(delta)) / (2 * a)
    raiz2 = (-b - math.sqrt(delta)) / (2 * a)
    return raiz1, raiz2  # retorna ambas as raízes

r1, r2 = bhaskara(a=0.5, b=14.3, c=45.0)
print(f'{r1 = }')
print(f'{r2 = }')
```

```
r1 = -3.5999999999999996
r2 = -25.0
```

Observe que `return` retorna uma `tupla` contendo dois valores do tipo `float`. Essa tupla é então desempacotada pelo código que chama a função, e cada valor retornado é atribuído a uma variável distinta (`r1` e `r2`).

¹Tecnicamente, toda função python tem um retorno. Se `return` não for usado dentro da função então por padrão o interpretador Python retornará `None`, que justamente é o objeto que indica a ausência de valor.



No código acima, a linha `import math` é uma declaração ao interpretador para importar o módulo `math`, já que `sqrt`, usada para calcular a raiz quadrada, não é uma função embutida (não está disponível por padrão). No entanto, ela faz parte da biblioteca padrão do Python, então não é necessário instalar nenhum pacote externo de terceiros.

Ainda nesse exemplo, introduzimos o comando `raise`, que lança uma exceção (geralmente associada a um erro) caso a função encontre raízes complexas, uma condição para a qual ela não foi projetada para lidar. Se essa exceção não for tratada² pelo código que chama a função, a execução do programa será interrompida.

2.2: Mutabilidade dos argumentos de funções

Outro aspecto importante sobre funções em Python é que os argumentos são sempre passados por referência (compartilhamento de objeto). Ou seja, os nomes das variáveis dentro da função apontam para os mesmos objetos que foram passados como argumento.

No entanto, há uma diferença de comportamento entre objetos imutáveis (como `int`, `float`, `str`, `bool` e `tuple`) e objetos mutáveis (como `list`, `dict`, `set` e classes definidas pelo usuário):

- Objetos imutáveis não podem ser alterados diretamente. Por isso, mesmo que a referência seja compartilhada, qualquer tentativa de modificação resulta na criação de um novo objeto. Na prática, eles se comportam como se fossem passados por cópia.
- Já objetos mutáveis podem ser alterados diretamente. Isso significa que modificações feitas dentro da função afetam o objeto original fora dela.

Abaixo vemos um exemplo de uma função que recebe uma variável imutável:

```
def quadrado(x):
    print(f"dentro da função antes da modificação: {id(x)=}, {x=}")
    x = x**2
    print(f"dentro da função após a modificação: {id(x)=}, {x=}")

a = 5
quadrado(a)
print(f"fora da função: {id(a)=}, {a=}")
```

```
dentro da função antes da modificação: id(x)=140706763580456, x=5
dentro da função após a modificação: id(x)=140706763581096, x=25
fora da função: id(a)=140706763580456, a=5
```

Fica evidente que a instrução `x = x**2` não modifica o objeto originalmente apontado por `x`, mas sim cria um novo objeto em uma nova posição da memória. A variável `x` passa, então, a referenciar esse novo objeto, ocultando (ou *shadowing*) a referência anterior dentro do escopo da função. A variável `a`, fora da função, permanece inalterada, continuando a referenciar o objeto original.

Agora, observe o comportamento distinto ao lidarmos com argumentos mutáveis:

```
def quadrado(x):
    print(f"dentro da função antes da modificação: {id(x)=}, {x=}")
    x[0] = x[0]**2
    x[1] = x[1]**2
    print(f"dentro da função após a modificação: {id(x)=}, {x=}")

a = [5, 3]
```

²O tratamento adequado de exceções em Python é realizado por meio da estrutura `try-except`, permitindo capturar e gerenciar erros de maneira controlada. No entanto, essa abordagem foge ao escopo desse texto introdutório.



```
quadrado(a)
print(f"fora da função: {id(a)=}, {a=}")
```

```
dentro da função antes da modificação: id(x)=2075912921088, x=[5, 3]
dentro da função após a modificação: id(x)=2075912921088, x=[25, 9]
fora da função: id(a)=2075912921088, a=[25, 9]
```

Como a lista é um objeto mutável, a função altera diretamente o conteúdo do objeto referenciado, mantendo o mesmo `id`. Isso mostra como mudanças feitas dentro da função refletem fora dela. O parâmetro ainda aponta para o mesmo objeto, agora modificado.

Compreender essa distinção é fundamental, pois muitas funções em Python podem não retornar um valor explícito, mas ainda assim modificar o estado de objetos mutáveis passados como argumento. Esse comportamento pode gerar efeitos colaterais significativos na execução do programa, especialmente quando não é antecipado pelo desenvolvedor.

Por outro lado, esse mecanismo também é uma ferramenta poderosa para evitar cópias desnecessárias de objetos grandes, permitindo que funções operem diretamente sobre estruturas complexas sem comprometer o desempenho ou o uso de memória.

Funções desse tipo, que alteram diretamente os dados aos quais têm acesso, são conhecidas como funções com efeitos colaterais (*side-effect functions*) e são bastante comuns em operações sobre estruturas de dados, como listas, dicionários ou objetos de classes definidas pelo usuário.

O oposto de uma *side-effect function* é uma função pura (*pure function*). Funções puras garantem que seus parâmetros de entrada sejam apenas lidos, sem qualquer modificação. Isso é possível mesmo quando se trabalha com objetos mutáveis, desde que se evite alterar seu estado. Para isso, é comum criar cópias dos objetos antes de aplicar transformações, o que preserva a integridade dos dados originais, mas implica em custos de duplicação do objeto, com maior uso de memória e tempo de processamento.

Em Python, a cópia de objetos pode ser realizada com as funções `copy()` ou `deepcopy()` do módulo `copy`, dependendo da profundidade desejada.

2.3: Recursão

Outro aspecto importante das funções é que elas permitem a implementação de algoritmos ou soluções com estrutura recursiva, ou seja, funções que chamam a si mesmas durante a execução.

Essa abordagem pode ser útil quando um problema pode ser naturalmente decomposto em subtarefas menores de mesma natureza. Entre os exemplos mais comuns estão a exploração de estruturas hierárquicas, geração de combinações, percurso de árvores binárias, execução de buscas binárias, dentre outros. Diversos algoritmos de cálculo numérico também fazem uso de estratégias recursivas, como o método da bisseção para encontrar raízes de funções, algoritmos de ordenação como o merge sort e a transformada rápida de Fourier (FFT).

Um exemplo clássico é a implementação de uma função que calcula o fatorial de um número, já que $n! = n \cdot (n - 1)!$:

```
def fatorial(n):
    if n == 0 or n == 1: # condição base (fim da recursão)
        return 1
    print(f"{n}! = {n} * {n - 1}!") # rastreamento da chamada
    return n * fatorial(n - 1) # chamada recursiva
```



```
print("início da recursão:")
resultado = fatorial(5)
print(f"5! = {resultado}")
```

```
início da recursão:
5! = 5 * 4!
4! = 4 * 3!
3! = 3 * 2!
2! = 2 * 1!
5! = 120
```

Outro exemplo clássico de recursão é a geração dos termos da sequência de Fibonacci, que se inicia com 0 e 1. Cada novo termo é calculado como a soma dos dois anteriores, formando uma progressão potencialmente infinita. Abaixo, temos uma implementação recursiva que utiliza acumuladores para calcular o *n*ésimo termo da sequência, além de exibir o rastreamento das chamadas recursivas:

```
def fibonacci(n, a=0, b=1):
    if n == 0: # caso base (fim da recursão)
        return a

    print(f"fibonacci(n={n - 1}, a={b}, b={a + b})") # rastreamento da chamada
    return fibonacci(n - 1, b, a + b) # chamada recursiva

print("início da recursão:")
resultado = fibonacci(5)
print(f"fibonacci(5) = {resultado}")
```

```
início da recursão:
fibonacci(n=4, a=1, b=1)
fibonacci(n=3, a=1, b=2)
fibonacci(n=2, a=2, b=3)
fibonacci(n=1, a=3, b=5)
fibonacci(n=0, a=5, b=8)
fibonacci(5) = 5
```

É importante lembrar que, como funções recursivas chamam a si mesmas, é necessário definir uma condição que interrompa o processo recursivo. Essa condição é conhecida como caso base.

As chamadas recursivas são empilhadas em memória até atingirem o caso base, e depois são resolvidas de trás pra frente (desempilhadas) através dos retornos às funções que as chamaram.

Além disso, é essencial que cada chamada recursiva contribua para a convergência para esse caso base. Ou seja, os argumentos devem ser atualizados de forma que o problema se torne progressivamente menor, até atingir a condição de parada.

Todo algoritmo recursivo pode ser reescrito de forma iterativa (a ser estudada no próximo capítulo). A versão iterativa geralmente é mais eficiente em termos de desempenho e uso de memória, embora, em alguns casos particulares, possa resultar em lógicas mais complexas. Na prática, ambas as abordagens têm seu espaço e são escolhidas conforme o contexto e a natureza do problema.

Capítulo 3

Estruturas de controle de fluxo

Até agora, os pequenos trechos de código que escrevemos, majoritariamente, seguiram uma sequência linear e bem definida, executando instruções em uma ordem preestabelecida, onde cada linha era processada sequencialmente e incondicionalmente, como seguir uma receita de bolo à risca.

No entanto, essa abordagem não é suficiente para representar comportamentos mais complexos, onde o próprio algoritmo precisa tomar decisões com base no estado atual. Por exemplo, em determinado ponto da execução, se uma condição específica for verdadeira, o programa pode seguir pelo caminho **A**; caso contrário, deve seguir pelo caminho **B**. Cada um desses caminhos corresponde a blocos de código distintos, que realizam ações diferentes para atingir um certo objetivo.

Além das decisões condicionais, os programas frequentemente precisam executar uma mesma ação várias vezes, sem que o programador precise escrever o mesmo código repetidamente. Além de ser impraticável, muitas vezes a lógica desejada exige que a decisão sobre quantas vezes executar uma tarefa seja tomada durante a própria execução do programa, baseada no estado atual.

Para isso, utilizamos as estruturas de repetição (ou loops), que permitem a execução contínua de um bloco de código enquanto certas condições forem atendidas, ou garantem que um bloco de código seja executado para todos os valores de uma lista, de forma iterativa. Dessa forma, o programa pode automatizar tarefas, tornando o código mais eficiente e adaptável.

Em qualquer programa de computador, controlar a ordem de execução das instruções é fundamental para garantir que a aplicação possa lidar com diferentes cenários, tomar decisões dinamicamente e repetir ações conforme necessário. Esse controle é realizado por meio das estruturas de controle de fluxo, que possibilitam modificar o comportamento de um programa baseado no estado atual, tornando-o mais flexível e inteligente.

Como aludido anteriormente, as estruturas de controle de fluxo podem ser classificadas em duas categorias: estruturas condicionais e estruturas de repetição. Sua implementação pode variar entre diferentes linguagens, e até mesmo dentro de uma mesma linguagem, podem existir múltiplas variantes para tornar seu uso mais conveniente em diferentes situações.

Por exemplo, em Python, decisões condicionais podem ser feitas com **if-else**, uma abordagem mais direta, ou com **match-case**, que facilita comparações mais organizadas entre



múltiplas possibilidades. Da mesma forma, repetições podem ser controladas com `while`, quando depende de uma condição, ou com `for`, que é ideal para percorrer sequências.

Essa flexibilidade permite que programadores escolham a estrutura mais adequada para cada contexto, tornando o código mais simples, expressivo e eficiente.

Nas próximas seções, exploraremos cada categoria detalhadamente, com exemplos para ilustrar seu funcionamento.

3.1: Estruturas condicionais

As estruturas condicionais permitem que um programa tome decisões durante sua execução com base em condições estabelecidas. Em Python, existem diferentes variantes dessa estrutura, tornando-a mais versátil para diversas situações. A abordagem mais comum é o uso de `if-else`, onde um bloco de código é executado apenas se uma condição for verdadeira. Para comparações múltiplas pode-se usar `if-elif-else` ou `match-case`, introduzido no Python 3.10, oferece uma alternativa estruturada, útil quando há diversas possibilidades de decisão.

3.1.1: `if-else`

Exemplo do uso de `if-else`:

```
idade = 20

if idade >= 18:
    print("Você é maior de idade.") # é executado se idade for maior ou igual a 18
else:
    print("Você é menor de idade.") # é executado se idade menor que 18
```

O comando `if` sempre espera uma expressão que resulte em um valor booleano, ou seja, `True` (verdadeiro) ou `False` (falso).

Quando a condição for avaliada como `True`, o bloco de código associado ao `if` será executado. Caso contrário, se houver um `else` (que é opcional), seu respectivo bloco será executado.

Cada bloco pode ter um tamanho arbitrário e conter múltiplas instruções, incluindo estruturas `if-else` aninhadas, permitindo expressar decisões mais complexas. Assim como nas funções, a delimitação de um bloco `if` é feita pela indentação.

No exemplo, dependendo do valor da variável `idade` a comparação `idade >= 18` retorna `True` ou `False`, e baseado nesse valor o `if` escolhe qual bloco de código executar, exibindo a mensagem apropriada.

Essas expressões booleanas geralmente envolvem o uso de operadores relacionais (`==`, `!=`, `>`, `<`, `>=`, `<=`), que podem ser combinados com operadores lógicos (`and`, `or`, `not`) para formar condições lógicas mais complexas. Além disso, o operador `is` verifica se duas variáveis referenciam o mesmo objeto na memória, diferindo do `==`, que compara os valores desses objetos. O operador `in` é útil para verificar se um determinado valor está presente em uma coleção, como uma lista. Outra função embutida `isinstance(obj, class)` é especialmente útil para verificar o tipo de um objeto dentro dessas expressões.

3.1.2: `if-elif-else`

Segue um exemplo de uma função que verifica o cadastro de um usuário para identificar um meio de contato válido. A prioridade é o email; caso não esteja disponível, o número de



telefone será retornado como alternativa.

```
def obter_contato(cadastro: dict) -> str:
    if 'email' in cadastro and cadastro['email'] != '':
        return cadastro['email']
    elif 'telefone' in cadastro and cadastro['telefone'] != '':
        return cadastro['telefone']
    else:
        return 'nenhum contato cadastrado'

usuario1 = {'nome': 'Fulano', 'email': 'fulano@ifsc.edu.br', 'telefone': '(48)99999-9999'}
usuario2 = {'nome': 'Beltrano', 'telefone': '(48)99999-8888'}
usuario3 = {'nome': 'Sicrano', 'telefone': ''}

print('contato usuario1:', obter_contato(usuario1))
print('contato usuario2:', obter_contato(usuario2))
print('contato usuario3:', obter_contato(usuario3))
```

```
contato usuario1: fulano@ifsc.edu.br
contato usuario2: (48)99999-8888
contato usuario3: nenhum contato cadastrado
```

3.1.3: match-case

A seguir temos um exemplo usando `match-case` para verificar se um determinado dia da semana é um final de semana ou um dia útil:

```
dia_da_semana = "sábado"

match dia_da_semana.lower():
    case "sábado" | "domingo":
        print("É fim de semana! Aproveite para descansar.")
    case "segunda" | "terça" | "quarta" | "quinta" | "sexta":
        print("É dia útil. Hora de trabalhar ou estudar!")
    case _: # executado caso não haja matching
        print("Dia inválido. Certifique-se de inserir um nome correto.")
```

Esse código verifica o valor da variável `dia_da_semana` e determina se o dia pertence ao final de semana ou se é um dia útil, comparando-o com as opções especificadas nos diferentes casos (`case`) dentro da estrutura `match-case`¹.

Cada `case` dentro de um `match` representa um possível caso de correspondência, permitindo que o programa escolha dinamicamente a execução adequada de apenas um deles.

O operador `|` possibilita agrupar múltiplos casos numa mesma cláusula, tornando o código mais legível e eficiente.

Além disso, a conversão para minúsculas (`lower()`) garante que a entrada do usuário funcione corretamente, independentemente de letras maiúsculas ou minúsculas.

3.2: Estruturas de repetição

As estruturas de repetição permitem que um programa execute um bloco de código múltiplas vezes de forma controlada, eliminando repetições manuais e tornando o código mais legível e organizado. Além de aprimorar a clareza, elas aumentam a eficiência e versatilidade do desenvolvimento ao possibilitar que o número de execuções seja definido dinamicamente durante a execução, adaptando-se às exigências da lógica que está sendo implementada.

¹em Python `match-case` é uma estrutura mais completa que não realiza apenas comparações de valores, mas sim *pattern matching*, que envolve conceitos de desestruturação e correspondência de padrões. Conceitos um pouco mais avançados que ficarão de fora desse tutorial introdutório.



Em Python, assim como na maioria das linguagens, há duas principais formas de implementar repetições: o comando `while`, ideal para execuções dependentes de uma condição dinâmica, e o comando `for`, mais adequado para percorrer elementos de uma sequência de maneira previsível. Cada uma dessas estruturas apresenta vantagens específicas e pode ser aplicada conforme a necessidade do programa, abrangendo desde a iteração sobre sequências ou coleções de dados até algoritmos que requerem monitoramento contínuo e definição dinâmica do critério de parada, como cálculos iterativos em modelos computacionais.

3.2.1: `while`

O comando `while` permite a repetição de um bloco de código enquanto uma condição for verdadeira, sendo especialmente útil em situações onde o número de iterações não pode ser determinado previamente. Essa abordagem possibilita a execução de tarefas que dependem de eventos externos, verificações contínuas ou condições de parada dinâmicas.

O exemplo a seguir apresenta um programa interativo no qual o usuário deve tentar adivinhar um número secreto (57). Caso acerte, vence o jogo e o programa é encerrado. Se errar, poderá continuar tentando até acertar.

```
numero_secreto = 57
print("Tente adivinhar o número entre 0 e 99.")
palpite = int(input("Digite seu palpite: "))

while palpite != numero_secreto:
    if palpite < numero_secreto:
        print("O número secreto é maior. Tente novamente!")
    else:
        print("O número secreto é menor. Tente novamente!")

    palpite = int(input("Digite seu palpite: "))

print("Parabéns! Você ganhou!")
```

Já no exemplo a seguir, o programa imprime no terminal os números naturais menores que 10. (valor definido pelo usuário):

```
contador = 0
while contador < 10:
    print(contador, end=' ')
    contador += 1
```

```
0 1 2 3 4 5 6 7 8 9
```

Nesse contexto, a variável `contador` desempenha o papel de variável de controle, sendo responsável por acompanhar e atualizar o estado da repetição a cada iteração.

Como o loop `while` apenas avalia uma expressão booleana, é necessário que essa variável seja inicializada, monitorada e modificada manualmente. Sua inicialização fora do laço e seu incremento em 1 a cada iteração é essencial para garantir o avanço do programa e evitar loops infinitos.

Esse processo de percorrer uma sequência pre-definida (ou até coleção de dados) é tão fundamental em programação que a maioria das linguagens, incluindo Python, oferece uma estrutura específica para essa finalidade. Com o comando `for`, o estado da iteração é gerenciado automaticamente, sem necessidade de controle manual pelo programador, tornando o código mais conciso e legível.



3.2.2: `for`

O comando `for` é utilizado principalmente para percorrer elementos de uma sequência, garantindo que cada item seja processado de maneira previsível.

O exemplo anterior pode ser reescrito substituindo o `while` por um `for` da seguinte forma:

```
for i in range(10):  
    print(i, end=' ')
```

```
0 1 2 3 4 5 6 7 8 9
```

Aqui, a variável `i` não precisa ser declarada ou incrementada manualmente, pois o próprio comando `for` atribui a ela cada valor dentro da sequência gerada por `range(10)`. Na primeira iteração, `i` recebe o valor `0`, na segunda `1`, e assim por diante, até atingir `9`. O valor `10` é não inclusivo, ou seja, o loop termina antes de alcançá-lo, e já que `10` gera uma sequência de `0` a `9`, temos exatamente `10` repetições do loop.

A estrutura `for` é particularmente útil ao trabalhar com listas, tuplas, dicionários e outras coleções de dados, permitindo percorrer diretamente seus elementos sem necessidade de manipular índices manualmente. Além disso, o `for` em Python é conceitualmente equivalente ao `for-each` presente em linguagens como Java e C#, pois permite acessar diretamente cada elemento de um iterável, tornando o processo de iteração mais intuitivo.

O exemplo a seguir demonstra isso:

```
nomes = ['Fulano', 'Beltrano', 'Sicrano']  
  
for nome in nomes:  
    print(f"Saudações, {nome}!")
```

```
Saudações, Fulano!  
Saudações, Beltrano!  
Saudações, Sicrano!
```

Aqui, o loop percorre diretamente os elementos da lista `nomes`, atribuindo cada valor individualmente à variável `nome` em cada iteração, sem exigir um índice explícito. Dessa forma, o processamento ocorre um a um, garantindo que cada elemento seja tratado separadamente. Essa abordagem torna o código mais claro, eliminando a necessidade de manipulação manual de índices e facilitando a interação direta com os dados da coleção.

Em situações onde é necessário acessar tanto o índice quanto o valor de cada elemento durante a repetição, o método `enumerate()` pode ser utilizado. Ele retorna pares de valores, onde cada elemento da coleção recebe seu respectivo índice de forma automática.

```
nomes = ['Fulano', 'Beltrano', 'Sicrano']  
  
for indice, nome in enumerate(nomes):  
    print(f"{indice}: Saudações, {nome}!")
```

```
0: Saudações, Fulano!  
1: Saudações, Beltrano!  
2: Saudações, Sicrano!
```

Python também oferece diversas funções embutidas para manipular iteradores. Por exemplo, `reversed()`, modifica o iterado para percorrer uma sequência de trás pra frente:

```
nomes = ['Fulano', 'Beltrano', 'Sicrano']  
  
for nome in reversed(nomes):  
    print(f"Saudações, {nome}!")
```



```
Saudações, Sicrano!  
Saudações, Beltrano!  
Saudações, Fulano!
```

Além disso, `zip()` possibilita a iteração simultânea sobre múltiplas coleções, associando pares de valores um a um a cada repetição:

```
nomes = ['Fulano', 'Beltrano', 'Sicrano']  
idades = [22, 35, 65]  
  
for nome, idade in zip(nomes, idades):  
    print(f"{nome} tem {idade} anos.")
```

```
Fulano tem 22 anos.  
Beltrano tem 35 anos.  
Sicrano tem 65 anos.
```

Assim como `list`, os `dict` também são `Iterables`. Ou seja, pode ser iterado (percorrido) em estruturas de repetição).

Por padrão, um `dict` é iterado por suas chaves:

```
pares = {'Fulano': 22, 'Beltrano': 35, 'Sicrano': 65}  
  
for key in pares:  
    print(f"A chave '{key}' está associada ao valor {pares[key]}.")
```

```
A chave 'Fulano' está associada ao valor 22.  
A chave 'Beltrano' está associada ao valor 35.  
A chave 'Sicrano' está associada ao valor 65.
```

Se quisermos iterar sobre pares chave-valor, podemos utilizar o método `.items()`, que retorna um iterador com tuplas no formato `(chave, valor)`:

```
pares = {'Fulano': 22, 'Beltrano': 35, 'Sicrano': 65}  
  
for key, value in pares.items():  
    print(f"A chave '{key}' está associada ao valor {value}.")
```

```
A chave 'Fulano' está associada ao valor 22.  
A chave 'Beltrano' está associada ao valor 35.  
A chave 'Sicrano' está associada ao valor 65.
```

Recursos adicionais relacionados a iteradores em Python podem ser acessados por meio do módulo padrão `itertools`.

Tanto o laço `while` quanto o `for` podem ser interrompidos a qualquer momento com o comando `break`. Esse mecanismo permite finalizar a execução do loop imediatamente, ignorando qualquer condição restante e continuando a execução do programa a partir da próxima instrução após o laço.

Capítulo 4

Exercícios

O desenvolvimento do raciocínio lógico e sua aplicação na programação são habilidades que se aprimoram, sobretudo, pela prática. Por isso, antes de explorarmos novos recursos, é essencial consolidarmos o que já estudamos.

Até agora, apresentamos pequenos trechos de código para ilustrar os principais mecanismos da linguagem. Python oferece uma ampla gama de funcionalidades, mas os conceitos abordados até aqui, como variáveis, tipos, expressões, funções e controle de fluxo, são fundamentais em qualquer linguagem de programação e já possibilitam a construção de diversas soluções.

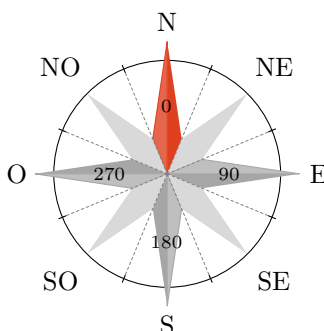
Os problemas propostos neste capítulo, embora em alguns casos se inspirem em desafios práticos reais, não têm como objetivo desenvolver soluções otimizadas para produção. Em vez disso, buscam fortalecer o raciocínio lógico, a capacidade de resolver problemas e a habilidade de transcrever soluções para o computador. Além disso, buscam desenvolver uma maior familiaridade com a linguagem, a IDE e o ecossistema que a envolve.

Questão 4.1: `bussola`

Implemente uma função `bussola(deg: float) -> str` que recebe um ângulo em graus e retorna o ponto cardinal ou colateral mais próximo como uma string (`'Sul'`, `'Sudeste'`, `'Leste'`, etc).

A função deve tratar corretamente valores negativos ou maiores que `360`, assegurando a normalização do ângulo por meio da aritmética modular.

Adote a convenção em que `0` graus corresponde ao Norte e `90` graus ao Leste.





Implemente o código:

```
def bussola(deg: float) -> str:
    # seu código aqui

#teste
print(bussola(20.7))
print(bussola(169.2))
print(bussola(-60.3))
print(bussola(868.5))
```

Saída esperada:

```
Norte
Sul
Noroeste
Sudeste
```

Questão 4.2: `desvio_padrao`

Escreva uma função que recebe como argumento uma lista numérica $\mathbf{x}$ de tamanho arbitrário e retorna o valor calculado do desvio padrão considerando probabilidades iguais para todos os elementos. Ou seja:

$$\sigma(\mathbf{x}) = \sqrt{\frac{1}{N-1} \sum_{n=0}^{N-1} (x_n - \bar{x})^2} \quad (4.1)$$

sendo $\bar{x}$ a média aritmética dos elementos de $\mathbf{x}$ e N o número de elementos de $\mathbf{x}$.

Compare o resultado de sua função com a função padrão `stdev()` do módulo `statistics`.

Questão 4.3: `login`

Escreva uma função para validar usuário e senha. Ao ser executado, o programa solicita ao usuário (via terminal) seu login e senha. Os valores são então comparados a um cadastro pré-definido (armazenado em um dicionário global). A função retorna `True` ou `False`, além de exibir uma mensagem no terminal informando se o acesso foi concedido ou negado.

```
def validacao(usuario: str, senha: str) -> bool:
    global CADASTROS
    # seu código aqui

# logins e senhas previamente cadastrados
CADASTROS = {
    'fulano' : '8g7&4A5',
    'beltrano': 'Hgr787@',
    'sicrano' : 'po78BA-'
}

# interrompe a execução e solicita usuário e senha
usuario = input("Digite seu login:")
senha = input("Digite sua senha:")

# chama a função de validação
acesso = validacao(usuario, senha)
```

Desafio:

Modifique o programa para permitir ao usuário três tentativas de acesso. Caso a senha seja digitada incorretamente três vezes consecutivas, essa senha deve ser permanentemente invalidada. Crie uma nova mensagem informando essa situação. A invalidação da senha de um usuário não deve afetar os demais.



Importante:

Tenha em mente que este exercício tem como objetivo explorar conceitos introdutórios de comparação e controle de fluxo. No entanto, por razões óbvias de segurança, senhas nunca devem ser armazenadas ou comparadas diretamente. Em aplicações reais, a validação de credenciais sempre envolve criptografia robusta e bancos de dados especializados, projetados para garantir a proteção das informações.

Questão 4.4: `separa_pares_impares`

Escreva uma função que recebe uma lista de números inteiros de tamanho arbitrário e retorne duas listas, a primeira contendo os elementos pares da lista original, e a segunda os ímpares.

```
def separa_pares_impares(lista):  
    # seu código aqui  
  
numeros = [5, 7, -9, 6, 4, 7, 9, 3, 8, 6]  
  
pares, impares = separa_pares_impares(numeros)  
print(f"pares: {pares}")  
print(f"ímpares: {impares}")
```

Resultado esperado:

```
pares: [6, 4, 8, 6]  
ímpares: [5, 7, -9, 7, 9, 3]
```

Desafio:

Reescreva a função adicionando anotação de tipos tanto para os parâmetros como para o retorno.

Questão 4.5: `seno`

Escreva uma função `seno(alpha: float) -> float` que receba um ângulo em radianos e retorne seu valor aproximado de seno, utilizando a seguinte série de Taylor:

$$\text{sen}(\alpha) = \alpha - \frac{\alpha^3}{3!} + \frac{\alpha^5}{5!} - \frac{\alpha^7}{7!} + \dots \quad (4.2)$$

A expansão deve ser truncada no primeiro termo em que:

$$\left| \frac{\alpha^k}{k!} \right| \leq 0.0001 \quad (4.3)$$

A série acima é convergente para ângulos pertencentes ao intervalo $[-\pi, +\pi]$. Portanto, antes de calcular a série, caso o ângulo informado esteja fora desse intervalo, ele deve ser mapeado para o intervalo de convergência.

Além da função de cálculo do seno, implemente um script de teste que compare os valores obtidos com os retornados pela função `math.sin()` do Python.



Questão 4.6: `produto_mais_vendido`

Crie um programa que receba um dicionário de produtos vendidos, contendo o nome do produto e a quantidade vendida. A função deve calcular e retornar uma tupla com o nome e a quantidade do produto mais vendido.

```
def produto_mais_vendido(dados: dict) -> tuple[str, int]:  
    # seu código aqui  
  
    vendas = {  
        'Notebook': 15,  
        'Smartphone': 32,  
        'Tablet': 12,  
        'Fones de ouvido': 45  
    }  
    nome, quantidade = produto_mais_vendido(vendas)  
    print(f"O produto mais vendido foi '{nome}' com {quantidade} vendas.")
```

Resultado esperado:

```
O produto mais vendido foi 'Fones de ouvido' com 45 vendas.
```

Questão 4.7: `alunos_aprovados`

Crie uma função que receba um dicionário de alunos e suas notas e retorne uma lista contendo apenas os nomes dos alunos aprovado (nota mínima 6.0).

```
def alunos_aprovados(dados):  
    # seu código aqui  
  
    alunos = {  
        'Alice': 8.5,  
        'Bruno': 5.2,  
        'Carlos': 9.0,  
        'Maria': 7.1,  
        'Pedro': 5.8  
    }  
    aprovados = alunos_aprovados(alunos)  
    print(f"Aprovados: {aprovados}")
```

Resultado esperado:

```
Aprovados: ['Alice', 'Carlos', 'Maria']
```

Desafio:

Substitua a linha contendo o `print` para uma chamada a uma nova função (a ser desenvolvida por você) que recebe a lista de aprovados e não retorna nada, mas que imprima os nomes dos aprovados de forma mais legível, com apenas um nome um por linha. Resultado esperado:

```
Aprovados:  
Alice  
Carlos  
Maria
```

Questão 4.8: `media_movel`

Escreva uma função `media_movel(serie: list[float], n: int) -> list[float]` que recebe como argumentos uma lista numérica `serie` de tamanho arbitrário e um inteiro `n`, indicando o número de elementos da janela de cálculo da média móvel a ser computada na lista `serie`. A função deve retornar uma nova série, na forma de uma lista de `float` com o mesmo



tamanho de `serie`, contendo os valores calculados da média móvel. Ou seja, para elementos cujo índice é maior ou igual a `n`:

$$y_k = \frac{1}{n} \sum_{i=k-n+1}^k x_i \quad , \quad k \geq n - 1 \quad (4.4)$$

onde x é a série original de entrada e y a que contem as médias moveis.

Preencha os elementos da serie de retorno cujos índices são menores que `n` com `float('nan')`.

Por exemplo, para a lista `[ 1.0, 3.0, 2.0, 4.0, 6.0, 2.0, 1.0 ]` e `n=3`:

$$\begin{aligned} x &= [\underbrace{1 \quad 3 \quad 2}_{y_2 = \frac{1+3+2}{3} = 2} \quad 4 \quad 6 \quad 2 \quad 1] \\ x &= [\quad 1 \quad \underbrace{3 \quad 2 \quad 4}_{y_3 = \frac{3+2+4}{3} = 3} \quad 6 \quad 2 \quad 1] \\ x &= [\quad 1 \quad 3 \quad \underbrace{2 \quad 4 \quad 6}_{y_4 = \frac{2+4+6}{3} = 4} \quad 2 \quad 1] \\ x &= [\quad 1 \quad 3 \quad 2 \quad \underbrace{4 \quad 6 \quad 2}_{y_5 = \frac{4+6+2}{3} = 4} \quad 1] \\ x &= [\quad 1 \quad 3 \quad 2 \quad 4 \quad \underbrace{6 \quad 2 \quad 1}_{y_6 = \frac{6+2+1}{3} = 3}] \\ y &= [\quad \cdot \quad \cdot \quad 2 \quad 3 \quad 4 \quad 4 \quad 3] \end{aligned}$$

```
def media_movel(serie: list[float], n: int) -> list[float]:
    # seu código aqui

print(media_movel([ 1.0, 3.0, 2.0, 4.0, 6.0, 2.0, 1.0 ], 3))
```

Resultado esperado:

```
[nan, nan, 2.0, 3.0, 4.0, 4.0, 3.0]
```

Questão 4.9: `ordena`

Escreva uma função `ordena(valores)`, que recebe uma lista de números inteiros de tamanho arbitrário e ordena seus elementos em ordem crescente. A função deve retornar duas listas: a primeira contendo os valores ordenados da lista original; a segunda contendo os índices dos elementos da lista original que correspondem à posição dos valores ordenados.

```
def ordena(valores: list[int]) -> tuple[list[int], list[int]]:
    # seu código aqui

vord, ids = ordena([9, 4, 3, 5, 1, 8])
print(f"vord: {vord}\n ids: {ids}")
```

Resultado esperado:

```
vord: [1, 3, 4, 5, 8, 9]
ids: [4, 2, 1, 3, 5, 0]
```



Não utilize a função `sort()` ou `sorted()` do Python. Implemente sua própria lógica de ordenação.

Depois de implementar seu próprio algoritmo, utilize as soluções embutidas da linguagem.

Questão 4.10: `procura`

Escreva uma função `procura(lista, valor)`, que recebe uma lista de números de tamanho arbitrário e um valor a ser buscado dentro dessa lista. A função deve retornar uma nova lista contendo os índices das ocorrências do valor na lista original.

Se o valor não for encontrado, a função deve retornar uma lista vazia.

```
def procura(lista, valor):  
    # seu código aqui  
  
ids = procura([2, 4, 8, 3, 0, 3, 5, 7], 3)  
print(ids)
```

Resultado esperado:

```
[3, 5]
```

Desafio:

Além de criar sua própria lógica para a busca, tente também resolver o problema utilizando o função embutida `filter()`. Para isso, você precisará aplicar uma função `lambda`, que não foi abordada neste tutorial. Pesquise sobre essa técnica e descubra como utilizá-la para filtrar os índices corretamente!

Questão 4.11: `balanceamento`

Escreva uma função `balanceamento(valores: list) -> tuple[list, list]` que receba uma lista numérica de tamanho arbitrário, calcule a média de seus valores e, com base nisso, retorne duas novas listas:

- a primeira contendo os elementos da lista original cujos valores são menores ou iguais à média;
- a segunda contendo os elementos estritamente maiores que a média.

Note que, em casos particulares, uma ou ambas as listas resultantes podem estar vazias.

Implemente

```
def balanceamento(valores: list) -> tuple[list, list]:  
    # seu código aqui  
  
l, g = balanceamento([2, 4, 8, 3, 0, 3, 5, 7])  
print(f"l = {l}")  
print(f"g = {g}")
```

Resultado esperado:

```
l = [2, 4, 3, 0, 3]  
g = [8, 5, 7]
```



Desafio:

Modifique a função para que ela aceite, opcionalmente, um segundo parâmetro com pesos:
`balanceamento(valores: list, pesos: list | None = None) -> tuple[list, list]`

Neste caso, `pesos` deve ser uma lista de mesmo tamanho que `valores`, representando o peso associado a cada elemento.

O objetivo agora é calcular a média ponderada dos valores, considerando os pesos fornecidos, e utilizá-la como critério para particionar os elementos em duas listas:

- a primeira lista deve conter os elementos cujo produto `valor * peso` seja menor ou igual à média ponderada;
- a segunda lista deve conter os elementos cujo produto `valor * peso` seja estritamente maior que a média ponderada.

Se o parâmetro `pesos` não for fornecido (`None`), a função deve atribuir peso `1` a todos os elementos, recuperando o comportamento original com média aritmética simples.

A função deve lançar uma exceção caso `valores` e `pesos` não tenham o mesmo tamanho ou se algum dos argumentos não seja uma lista numérica.

Implemente:

```
def balanceamento(valores: list, pesos: list | None = None) -> tuple[list, list]:
    # seu código aqui

valores = [8, 3, 11, 6, 4, 9]
pesos = [1.0, 0.8, 1.2, 0.9, 1.1, 1.0]

l, g = balanceamento(valores, pesos)

print(f"l = {l}")
print(f"g = {g}")
```

Resultado esperado:

```
l = [3, 6, 4]
g = [8, 11, 9]
```

Questão 4.12: `tictactoe`

Escreva uma função `eval_move(board_state, next_move)` que processe a lógica do jogo tic-tac-toe (jogo da velha) e informa o resultado da partida.

O primeiro argumento recebido pela função (`board_state`) é uma matriz 3×3 (implementada na forma de `list[list[int]]`) que contém o estado atual do tabuleiro. Elementos da matriz contendo `0` indicam que aquela posição ainda não foi jogada. Posições da matriz contendo `1` indicam que ela foi tomada pelo jogador `X`, enquanto posições contendo `-1` foram tomadas pelo jogador `O`.

Valores diferentes de `-1`, `0` e `1` constituem um estado inválido do tabuleiro. Outras formas de estados inválidos também devem ser considerados. Por exemplo, muitas posições tomadas por um mesmo jogador ou o jogador `O` com mais posições tomadas que `X` (considerando que `X` sempre inicia jogando). Abaixo são apresentados alguns valores de `board_state` e o que cada um deles representa.



```
# valid state
board_state = [
    [ 1, -1, 0],
    [ 0, 1, 1],
    [-1, 0, -1]
]
```

X	O	
	X	X
O		O

```
# invalid state
board_state = [
    [ 1, -1, 0],
    [ 0, -1, 1],
    [-1, 0, -1]
]
```

X	O	
	O	X
O		O

```
# player x wins!
board_state = [
    [ 1, -1, -1],
    [ 0, 1, 1],
    [-1, 0, 1]
]
```

X	O	O
	X	X
O		X

```
# draw!
board_state = [
    [ 1, -1, 1],
    [-1, -1, 1],
    [ 1, 1, -1]
]
```

X	O	X
O	O	X
X	X	O

O segundo argumento recebido (`next_move`) é uma tupla de dois inteiros (`l,c`), indicando respectivamente a linha e a coluna jogada pelo próximo jogador. Valores de `l` e `c` diferentes de `0`, `1` e `2` constituem uma tentativa de jogada inválida. Também é uma jogada inválida qualquer posição (`l,c`) previamente ocupada.

A função deve inferir qual dos jogadores (`X` ou `O`) está realizando a jogada `next_move` a partir do estado do tabuleiro (`board_state`), garantindo a alternância entre os jogadores. O jogador `X` joga primeiro.

Uma vez validado o estado atual, o jogador da vez e a posição a ser jogada por este, a função `eval_move()` deve modificar o estado do tabuleiro executando a jogada solicitada (*side-effect function*). Se a jogada solicitada ou a posição prévia do tabuleiro for inválida, a função mantém o `board_state` inalterado.

Alem de modificar `board_state`, a função também retorna um valor (`outcome`), indicando se a jogada ou posição são inválidas, se houve um vencedor, se houve empate ou se o jogo ainda não terminou. O retorno `outcome` deve ser um inteiro que codifica essas situações da seguinte forma:

- `-4` se a posição do tabuleiro passado como argumento for ilegal;
- `-2` se a próxima jogada for ilegal;
- `0` se a jogada foi executada com sucesso, mas ainda não houve um ganhador;
- `1` se a jogada foi executada com sucesso e o jogador `X` venceu;
- `2` se a jogada foi executada com sucesso e o jogador `O` venceu;
- `3` se a jogada foi executada com sucesso, mas o jogo terminou em empate.

Complete:

```
def eval_move(board_state: list[list[int]], next_move: tuple[int, int]) -> int:
    #seu código aqui

board_state = [
    [ 1, -1, 0],
    [ 0, 0, 1],
    [-1, 0, 0]
]
outcome = eval_move(board_state, (1, 1))
print(f"board_state = {board_state}, outcome = {outcome}")

outcome = eval_move(board_state, (2, 2))
print(f"board_state = {board_state}, outcome = {outcome}")

outcome = eval_move(board_state, (0, 1))
print(f"board_state = {board_state}, outcome = {outcome}")
```



```
outcome = eval_move(board_state, (1, 0))  
print(f"{board_state = }, {outcome = }")
```

Resultado esperado:

```
board_state = [[1, -1, 0], [0, 1, 1], [-1, 0, 0]], outcome = 0  
board_state = [[1, -1, 0], [0, 1, 1], [-1, 0, -1]], outcome = 0  
board_state = [[1, -1, 0], [0, 1, 1], [-1, 0, -1]], outcome = -2  
board_state = [[1, -1, 0], [1, 1, 1], [-1, 0, -1]], outcome = 1
```

Note que a função deve realizar várias ações:

- Validação do estado atual do tabuleiro;
- Determinação do próximo jogador;
- Verificação da próxima jogada;
- Aplicação da jogada no tabuleiro;
- Avaliação do resultado (vitória, empate ou jogo em andamento).

Convém dividir a implementação da função `eval_move` em funções menores, dedicadas a cada uma dessas sub-tarefas (nomeie essas funções com nomes significativos).

Questão 4.13: `tictactoe` (continuação)

Em <https://github.com/j-Lago/pyTicTacToe> você encontrará o repositório de um código python para uma interface gráfica do jogo da questão anterior. Ao executar `main.py`, o programa abrirá uma janela gráfica com um tabuleiro tic-tac-toe vazio. Ao clicar com o botão esquerdo do mouse em uma das posições do tabuleiro, o programa chamará a função `eval_move()` do arquivo `eval_move.py` (o arquivo existe, mas a implementação da função está faltando).

Edite o arquivo `eval_move.py`, incluindo a sua implementação para a função desenvolvida na questão anterior, execute `main.py` e verifique se sua função realmente implementa a lógica do jogo.

Desafio:

Pesquise sobre `Enums` e incorpore-os na representação de `outcome` para deixar o código mais legível e organizado.

Capítulo 5

Classes

Além dos tipos primitivos e coleções introduzidos no Capítulo 1, podemos ainda criar nossos próprios tipos personalizados.

Uma classe descreve um tipo composto que reúne, em um único objeto, um conjunto coerente de variáveis (atributos) e as funções que operam sobre elas (métodos), para a manipulação desses dados.

Embora as classes sejam um dos pilares da Programação Orientada a Objetos (OOP), viabilizando conceitos como encapsulamento, abstração, herança e polimorfismo, este texto introdutório não se propõe a discutir, muito menos advogar por OOP como abordagem universal.

Ainda assim, o uso de classes pode melhorar significativamente a organização do código, mesmo sem aderir estritamente aos princípios da OOP.

Além disso, compreender a estrutura básica de classes em Python é essencial, especialmente para entender e utilizar pacotes especializados, que serão explorados em capítulos futuros.

Nada melhor do que demonstrar na prática as vantagens do uso de classes. Para isso, vamos revisitar um dos exemplos apresentados no Capítulo 3, onde criamos uma função para buscar o contato de um usuário a partir de seu cadastro.

O código original é replicado abaixo:

```
def obter_contato(cadastro: dict) -> str:
    if 'email' in cadastro and cadastro['email'] != '':
        return cadastro['email']
    elif 'telefone' in cadastro and cadastro['telefone'] != '':
        return cadastro['telefone']
    else:
        return 'nenhum contato cadastrado'

usuario1 = {'nome': 'Fulano', 'email': 'fulano@ifsc.edu.br', 'telefone': '(48)99999-9999'}
usuario2 = {'nome': 'Beltrano', 'telefone': '(48)99999-8888'}
usuario3 = {'nome': 'Sicrano', 'telefone': ''}

print('contato usuario1:', obter_contato(usuario1))
print('contato usuario2:', obter_contato(usuario2))
print('contato usuario3:', obter_contato(usuario3))
```

```
contato usuario1: fulano@ifsc.edu.br
contato usuario2: (48)99999-8888
contato usuario3: nenhum contato cadastrado
```



Note que a função `obter_contato()` possui um único parâmetro `cadastro`, que é um dicionário com algumas chaves esperadas: `'nome'`, `'telefone'` e `'email'`.

O uso de um dicionário, nesse contexto, foi a forma encontrada para agrupar as informações relacionadas a um mesmo indivíduo, evitando que tais dados fiquem soltos, espalhados em variáveis independentes. Como os valores associados às chaves `'nome'`, `'telefone'` e `'email'` estão logicamente vinculados, faz sentido organizá-los sob uma mesma estrutura.

Essa abordagem apresenta fragilidades e limitações, uma vez que a função `obter_contato()` depende da existência de chaves específicas no dicionário fornecido para funcionar corretamente. O código não prevê mecanismos estruturados para lidar com dados inconsistentes ou ausentes, delegando ao chamador a responsabilidade de assegurar a organização e a completude do dicionário fornecido. O chamador, de certa forma, precisa conhecer aspectos internos da função.

Uma forma mais estruturada de lidar com essas questões é utilizar classes, que encapsulam dados (`nome`, `telefone`, `email`) e funcionalidade (`obter_contato()`) em um único objeto.

Poderíamos refatorar o programa criando um novo tipo personalizado chamado `Usuario`:

```
1 # definição da classe
2 class Usuario:
3     # o método __init__ é executado ao criar um novo objeto
4     def __init__(self, nome: str, telefone: str = '', email: str = ''):
5         self.nome = nome
6         self.telefone = telefone
7         self.email = email
8
9     def contato(self) -> str:
10        if self.email:
11            return self.email
12        elif self.telefone:
13            return self.telefone
14        else:
15            return 'nenhum contato cadastrado'
16
17 # criando instâncias da classe Usuario
18 usuario1 = Usuario(nome='Fulano', email='fulano@ifsc.edu.br', telefone='(48)99999-9999')
19 usuario2 = Usuario(nome='Beltrano', telefone='(48)99999-8888')
20 usuario3 = Usuario(nome='Sicrano')
21
22 # utilizando o método contato() e apresentando os resultados
23 print('contato usuario1:', usuario1.contato())
24 print('contato usuario2:', usuario2.contato())
25 print('contato usuario3:', usuario3.contato())
```

```
contato usuario1: fulano@ifsc.edu.br
contato usuario2: (48)99999-8888
contato usuario3: nenhum contato cadastrado
```

Vamos explorar as partes fundamentais desse código linha por linha.

Na **linha 2** temos a declaração que estamos definindo uma nova `class` chamada `Usuario`¹.

Essa definição consiste em dois métodos, ou duas funções associadas à classe: `__init__()` (**linha 4**) e `contato()` (**linha 9**).

O método `__init__()` é o construtor da classe, que é executado automaticamente ao criarmos um novo objeto do tipo `Usuario` (o que é feito na **linha 18**). Esse processo é conhecido como instanciação da classe, e resulta na criação de um objeto em memória. Enquanto a classe é

¹Diferente da convenção utilizada para funções e variáveis, que geralmente seguem o estilo *snake_case* (`exemplo_variavel`), nomes de classes costumam ser escritos em *CamelCase* (`NomeDaClasse`).



apenas uma definição da estrutura e do comportamento dos objetos (um *blueprint*), a instância representa a consolidação de um objeto desse tipo em memória, com valores próprios.

Assim como funções, os métodos podem ter parâmetros que podem ser obrigatórios ou opcionais (recebendo valores padrão). No caso de `__init__()`, os parâmetros são `self`, `nome`, `email` e `telefone`.

A principal diferença desse método quando comparado a uma função padrão, é a presença do parâmetro `self`, que é uma referência à própria instância do objeto². Não precisamos (nem devemos) fornecer `self` manualmente a um método, já que o Python faz isso de forma automática e disponibiliza a referência `self` aos métodos da classe justamente para que estes possam manipular os dados relativos à instância.

Na **linha 5**, criamos o atributo `self.nome`, atribuindo a ele o valor do parâmetro `nome`.

Um atributo é uma variável interna da instância da classe, responsável por armazenar informações específicas de cada objeto criado. São esses atributos que compõem o objeto.

Embora o atributo e o parâmetro tenham o mesmo nome (o que é uma prática comum, mas não necessária), eles são variáveis distintas. O parâmetro `nome` é passado ao método `__init__()` no momento da criação de um novo objeto da classe `Usuario`, e seu valor é armazenado internamente no atributo `self.nome`, garantindo que essa informação fique acessível ao longo da vida útil do objeto.

Além de `nome`, são criados mais dois atributos para armazenar `email` e `telefone`. Esses três atributos formam a estrutura de dados armazenados em memória para um objeto da classe `Usuario`.

Pulando para a **linha 18**, que na realidade é a primeira a ser executada (já que as anteriores pertencem apenas à definição da classe `Usuario`), temos a criação da variável `usuario1`.

À direita do operador de atribuição (`=`) há uma chamada à classe `Usuario()`, o que significa que um novo objeto do tipo `Usuario` está sendo instanciado.

Ao chamar `Usuario(nome='Fulano', email=...)`, os argumentos fornecidos (`nome`, `email` e `telefone`) são passados para o método `__init__()`, que então atribui esses valores aos atributos internos do objeto. Isso resulta na criação de uma instância única, armazenada na variável `usuario1`, contendo todas as informações fornecidas.

Nas **linhas 19 e 20**, criamos mais duas instâncias da classe `Usuario`, com valores distintos. Cada instância da classe representa um objeto único na memória, contendo suas próprias informações de `nome`, `telefone` e `email`, mas possuem a mesma estrutura definida pela classe.

Na **linha 23**, dentro do comando `print()`, temos a chamada `usuario1.contato()`, que executa o método `contato()` definido na **linha 9**.

Um detalhe importante é que, na definição de `contato(self)`, declaramos um parâmetro `self`, mas não o fornecemos explicitamente na chamada do método.

Isso ocorre porque o Python passa `self` automaticamente, garantindo que o método seja

²O nome `self` é apenas uma convenção, e sua função de autoreferência à instância do objeto está diretamente relacionada à posição do parâmetro, que deve sempre ser o primeiro em qualquer método. Independentemente do nome utilizado, esse primeiro parâmetro representa o próprio objeto, permitindo o acesso e a manipulação de seus atributos e métodos. Contudo, usar `self` é altamente recomendado, pois é uma prática amplamente reconhecida e melhora a legibilidade do código.



chamado sobre a instância correta. Ou seja, a chamada `usuario1.contato()` equivale internamente a `contato(usuario1)`, onde `self` recebe `usuario1` como argumento.

Note que o encapsulamento dos dados e funcionalidades contribui para a organização e legibilidade do código. As instâncias de classes geralmente são objetos mutáveis, como os abordados no Capítulo 1. Na realidade, em Python, todos os tipos de dados são implementados como classes, até mesmo os tipos primitivos como inteiros.

Diferente de outras linguagens como Java, C++, C# e Rust, onde existe um sistema explícito de controle de acesso (como `private`, `protected` e `public`), em Python todos os membros de uma classe são públicos por padrão.

Isso significa que qualquer parte do código que tenha acesso a uma instância da classe pode ler e modificar diretamente seus atributos, além de chamar qualquer método definido na classe.

Embora Python não imponha um mecanismo formal de privacidade, existe uma convenção para indicar atributos internos³.

O exemplo a seguir demonstra esse acesso a atributos:

```
class Usuario:
    def __init__(self, nome: str, telefone: str = '', email: str = ''):
        self.nome = nome
        self.telefone = telefone
        self.email = email

usuario = Usuario(nome='Fulano', email='fulano@ifsc.edu.br', telefone='(48)99999-9999')
print(f"{usuario.nome}, {usuario.email}, {usuario.telefone}") # lendo atributos

usuario.email = 'fulano_de_tal@ifsc.edu.br' # modificando atributo email
usuario.telefone = '(11)88888-9999' # modificando atributo telefone
print(f"{usuario.nome}, {usuario.email}, {usuario.telefone}") # lendo atributos
```

```
Fulano, fulano@ifsc.edu.br, (48)99999-9999
Fulano, fulano_de_tal@ifsc.edu.br, (11)88888-9999
```

Em outro exemplo, poderíamos criar uma segunda classe denominada `Cadastro`, que agrupa diversos usuários e oferece funcionalidades sobre essa coleção.

```
1 class Usuario:
2     def __init__(self, nome: str, email: str = '', telefone: str = ''):
3         self.nome = nome
4         self.email = email
5         self.telefone = telefone
6
7     def imprimir_contato(self) -> str:
8         if self.email:
9             return self.email
10        elif self.telefone:
11            return self.telefone
12        else:
13            return 'nenhum contato cadastrado'
14
15 class Cadastro:
16     def __init__(self):
17         self.usuarios = []
18
19     def adicionar_usuario(self, usuario: Usuario):
20         self.usuarios.append(usuario)
21
```

³Convencionou-se utilizar um underscore (`_`) antes do nome do atributo para indicar que ele não deve ser acessado diretamente (ex: `self._informacao_privada`). Apesar de ainda ser possível modificar esses atributos externamente, essa prática sinaliza que o atributo é de uso interno, recomendando que sua manipulação ocorra apenas dentro da própria classe por seus métodos. IDEs e ferramentas de desenvolvimento costumam alertar sobre tentativas de acesso direto a esses atributos, reforçando essa boa prática.



```
22     def procurar_usuario(self, nome: str) -> Usuario | None:
23         for usuario in self.usuarios:
24             if usuario.nome == nome:
25                 return usuario
26         return None
27
28     def imprimir_contato(self, nome: str):
29         selecionado = cadastro.procurar_usuario(nome)
30         if isinstance(selecionado, Usuario):
31             print(f"O contato de {selecionado.nome} é: {selecionado.imprimir_contato()}")
32         else:
33             print(f"Usuário '{nome}' não encontrado.")
34
35 cadastro = Cadastro()
36 cadastro.adicionar_usuario(Usuario('Fulano', 'fulano@ifsc.edu.br', '(48)99999-9999'))
37 cadastro.adicionar_usuario(Usuario('Beltrano', telefone='(48)99999-8888'))
38 cadastro.adicionar_usuario(Usuario('Cicrano'))
39
40 cadastro.imprimir_contato("Beltrano")
```

As classes em Python oferecem diversas funcionalidades avançadas, como herança, sobrecarga de operadores, métodos mágicos, polimorfismo, abstração, propriedades e metaclasses, que ampliam o suporte à programação orientada a objetos.

Embora sejam conceitos importantes, este tutorial introdutório foca nos princípios básicos. O aprofundamento nesses temas fica a critério do leitor, que poderá explorá-los de forma gradativa, em um momento oportuno, conforme sua necessidade e evolução no aprendizado.

Para praticar os conceitos de classes, refatore o código da questão [tictactoe](#), transformando o parâmetro `board_state` em um atributo e a função `eval_move()` método da classe `TicTacTtoe`.

Capítulo 6

NumPy, Matplotlib e Pandas

A lista (`list`) em Python é uma estrutura de dados altamente versátil: cada elemento pode conter qualquer tipo de objeto, independentemente do tamanho ou tipo, e a estrutura pode ser modificada e redimensionada dinamicamente. No entanto, toda essa generalidade e flexibilidade tem um custo, especialmente em termos de desempenho.

Diferente de outras linguagens, o Python não possui um tipo embutido para representar `arrays` no sentido tradicional, ou seja, coleções de dados homogêneos com tamanho fixo. Esse tipo de estrutura, embora bem mais restrita que uma lista, permite otimizações tanto no uso de memória quanto na performance.

Isso torna-se particularmente crítico quando lidamos com grandes volumes de dados numéricos em aplicações científicas, como regressão, otimização, álgebra linear os demais métodos utilizados em computação científica. Nessas situações, a eficiência no processamento é essencial, exigindo estruturas de dados mais especializadas e performáticas que as listas tradicionais do Python.

A solução mais amplamente adotada para esse desafio em Python é o uso da biblioteca `numpy`, que, embora não venha incluída por padrão na instalação da linguagem, consolidou-se como o padrão de fato no meio científico para computação numérica. Complementarmente, as bibliotecas `pandas` e `matplotlib` são amplamente utilizadas, respectivamente, para a manipulação e análise de dados estruturados (como tabelas), bem como para a visualização de dados por meio da geração de gráficos (`plots`).

Como `numpy`, `matplotlib` e `pandas` não fazem parte da biblioteca padrão, é necessário instalá-los manualmente. Isso pode ser feito utilizando o `pip`, o gerenciador de pacotes oficial do Python. Para realizar a instalação, basta executar o seguinte comando no terminal:

```
C:\curso_python> pip install numpy matplotlib pandas
```

6.1: NumPy

A biblioteca `numpy` fornece a estrutura de dados `ndarray`, que permite representar `arrays` multidimensionais de forma compacta e eficiente. Ela também disponibiliza um vasto conjunto de funções vetorizadas otimizadas, capazes de aplicar operações matemáticas diretamente sobre todo o `array`, dispensando o uso de laços explícitos que podem ser pouco performáticos



em Python. Essa abordagem vetorizada resulta em cálculos muito mais rápidos, especialmente ao lidar com grandes volumes de dados.

Uma das aplicações fundamentais da `numpy` é na representação e manipulação de matrizes. O exemplo a seguir ilustra esse uso:

```
1 import numpy as np
2
3 A = np.array([[2, 3],      # define matriz 2 x 2
4               [-4, 1]])
5 x = np.array([[3],        # define matrix 2 x 1
6               [1]])
7 b = np.array([[-3],       # define matrix 2 x 2
8               [8]])
9
10 y = A @ x + b    # operação de multiplicação e soma matricial
11 print(y)
```

```
[[ 6]
 [-3]]
```

Na **linha 1**, declaramos que utilizaremos o pacote `numpy`, e o faremos por meio do *alias* `np`. Ou seja, para não repetirmos `numpy` diversas vezes ao longo do código, atribuímos a ele o nome abreviado `np`.

Na **linha 3**, criamos a matriz **A** de dimensão 2×2 utilizando `array`. Já nas **linhas 5 e 7**, definimos os vetores coluna **x** e **b**, ambos com dimensão 2×1 .

E na **linha 10** realizamos a operação matricial $y = A \cdot x + b$. Observe que o operador `@` realiza o produto matricial, conforme definido na álgebra linear. Já o operador `*` também pode ser utilizado com arrays `numpy`, mas nesse caso representa a multiplicação elemento a elemento, e não a multiplicação de matrizes.

Por fim, na **linha 10**, realizamos a operação matricial $y = A \cdot x + b$. Note que o operador `@` realiza o produto matricial, conforme definido na álgebra linear. Já o operador `*` também pode ser usado com `arrays` do `numpy`, mas nesse caso ele representa a multiplicação elemento a elemento, e não o produto de matrizes.

Além das operações matriciais, o `numpy` também se destaca ao lidar com sequências numéricas unidimensionais, graças à sua poderosa implementação de vetorização, que simplifica tanto a escrita do código quanto sua performance.

No exemplo a seguir, usamos um `array` para representar uma sequência numérica e, com isso, podemos usar vetorização para aplicar uma expressão matemática a cada elemento dessa sequência:

```
1 import numpy as np
2
3 x = np.linspace(0, 3, 7)
4 y = x**2 - 2*x
5
6 print(f"x = {x}")
7 print(f"y = {y}")
```

```
x = array([0. , 0.5, 1. , 1.5, 2. , 2.5, 3. ])
y = array([-0. , -0.75, -1. , -0.75, 0. , 1.25, 3.  ])
```

Na **linha 3**, utilizamos o método `linspace` da biblioteca `numpy` para criar um `array` unidimensional (um vetor linha) contendo **7** valores do tipo `float`, igualmente espaçados no intervalo fechado de **0** a **3**, ou seja, incluindo os dois extremos do intervalo.



Na **linha 4**, exploramos as funcionalidades de vetorização da biblioteca `numpy` para aplicar a expressão matemática diretamente a todos os elementos de `x`. O resultado é um novo `array y`, de mesmo tamanho, cujos valores correspondem à aplicação da expressão $y_n = x_n^2 - 2x_n$ em cada elemento x_n do `array x`, sem a necessidade de laços explícitos.

Nos exemplos anteriores, usamos `np.array([...])` para criar um `array` a partir de uma lista e `np.linspace()` para gerar valores igualmente espaçados dentro de um intervalo. Além dessas abordagens, a biblioteca `numpy` oferece outras funções bastante úteis para a criação de `arrays`, como:

- `np.zeros(shape)` – cria um `array` preenchido com zeros.
- `np.ones(shape)` – cria um `array` preenchido com uns.
- `np.full(shape, valor)` – cria um `array` preenchido com um valor específico.
- `np.eye(n)` – gera a `matriz` identidade de ordem `n`.
- `np.arange(início, fim, passo)` – semelhante à função `range`, mas para `array`.
- `np.random.rand(shape)` – cria um `array` com valores aleatórios no intervalo `[0, 1)`.

Os exemplos apresentados acima têm como objetivo apenas introduzir a `numpy`. Ela oferece uma ampla gama de funcionalidades não abordadas, incluindo muitas outras operações vetoriais, manipulação de `arrays` multidimensionais, transformações estruturais, mapeamento e interpolação de valores, entre muitos outros recursos voltados à computação numérica. Recomenda-se que o leitor explore essas funcionalidades conforme sua necessidade. A documentação oficial da biblioteca está disponível em: <https://numpy.org/doc>.

Além disso, recomenda-se ao leitor explorar a biblioteca `SciPy`, que complementa e expande as funcionalidades do `numpy` ao disponibilizar ferramentas especializadas para áreas como álgebra linear, equações diferenciais, integração numérica, otimização, interpolação, transformadas e processamento de sinais. A documentação oficial da biblioteca está disponível em: <https://docs.scipy.org/doc/scipy/>.

6.2: Matplotlib

Outra biblioteca particularmente útil em aplicações de computação científica é a `matplotlib`. Com uma pequena modificação no exemplo anterior, podemos utilizá-la para gerar um gráfico que representa `x` e `y` no plano cartesiano.

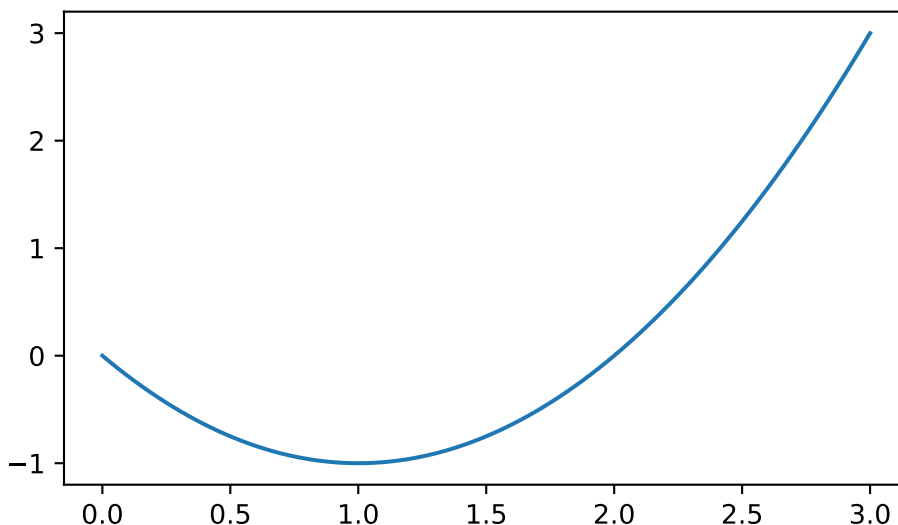
```
import numpy as np
import matplotlib.pyplot as plt

# gera valores para os eixos x e y do gráfico
x = np.linspace(0, 3, 100)
y = x**2 - 2*x

# plota o gráfico
plt.plot(x, y)
plt.show()
```



Esse código gera e exibe a figura:



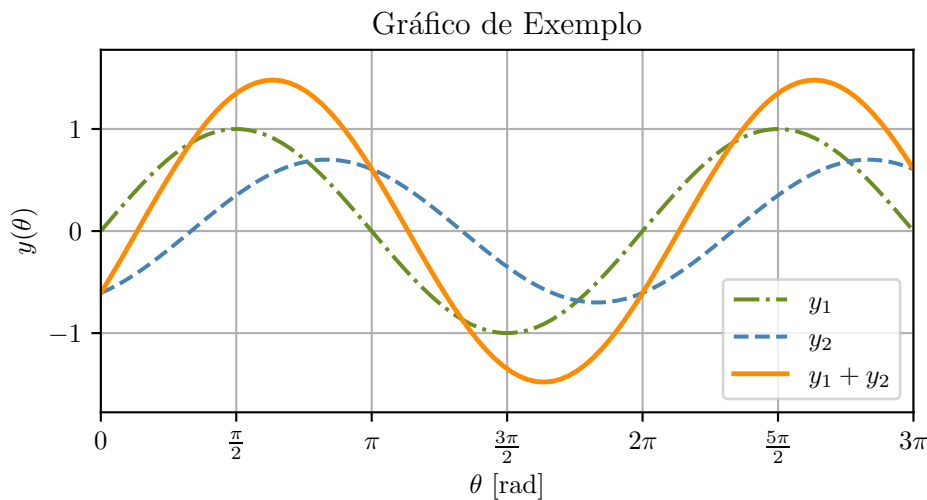
A figura gerada exibe o gráfico construído a partir dos pares ordenados (x_n, y_n) . Embora seja uma visualização simples, a biblioteca `matplotlib` oferece uma ampla variedade de funções auxiliares para personalizar o aspecto visual do gráfico.

O exemplo a seguir demonstra algumas dessas funcionalidades:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # gera pontos para os eixo x e y
5 theta = np.linspace(0, 3*np.pi, 100)
6 y1 = np.sin(theta)
7 y2 = 0.7 * np.sin(theta - np.pi/3)
8 y3 = y1 + y2
9
10 # descomete caso queira fidelidade de fontes com Latex (necessário instalar Latex)
11 plt.rcParams.update({'text.usetex': True, 'font.family': 'serif'})
12
13 plt.figure(figsize=(5, 2.8)) # define o largura e altura da figura em polegadas
14 plt.plot(theta, y1, color='olivedrab', linestyle='-.', label=r'$y_1$') # plota y1
15 plt.plot(theta, y2, color='steelblue', linestyle='--', label=r'$y_2$') # plota y2
16 plt.plot(theta, y3, color='darkorange', linewidth=1.8, label=r'$y_1+y_2$') # plota y3
17 plt.title('Gráfico de Exemplo') # adiciona título ao gráfico
18 plt.xlabel(r'$\theta$ [rad]') # adiciona identificação do eixo x
19 plt.ylabel(r'$y(\theta)$') # adiciona identificação do eixo y
20 plt.xlim(theta[0], theta[-1]) # define os limites do eixo x
21 plt.ylim(1.2*y3.min(), 1.2*y3.max()) # define os limites do eixo y
22 plt.grid(True) # exibe grade
23 plt.legend() # exibe legenda (valores atribuidos a plabel no plot)
24 plt.tight_layout() # ajusta o grafico gerado aos limites da figura
25
26 # define os ticks do eixo x manualmente como múltiplos de  $\pi$ 
27 xticks = [0, np.pi/2, np.pi, 3*np.pi/2, 2*np.pi, 5*np.pi/2, 3*np.pi]
28 xtick_labels = [r'$0$', r'$\frac{\pi}{2}$', r'$\pi$', r'$\frac{3\pi}{2}$',
29                 r'$2\pi$', r'$\frac{5\pi}{2}$', r'$3\pi$']
30 plt.xticks(xticks, xtick_labels)
31
32 # exporta a figura gerada para um arquivo externo do tipo .pdf
33 plt.savefig(r"seno.pdf", format="pdf")
34
35 # exibe a figura na tela
36 plt.show()
```



Saída do programa:



Mais informações sobre os recursos de `matplotlib` podem ser encontradas na documentação oficial da biblioteca, disponível em: <https://matplotlib.org/stable/contents.html>.

6.3: Pandas

Assim como a biblioteca `numpy` fornece uma estrutura para representar `arrays` de elementos homogêneos, a biblioteca `pandas` introduz a estrutura `DataFrame`, destinada a representar e manipular tabelas.

Um `pandas.DataFrame` é, de forma simplificada, uma tabela com linhas e colunas rotuladas, semelhante a uma planilha do `Excel`.

Veja um exemplo simples:

```
import pandas as pd

df = pd.DataFrame(data={'nome': ['Fulano', 'Beltrano', 'Sicrano'],
                        'documento': [5342, 1876, 1872],
                        'idade': [25, 15, 43],
                        'habilitado': [False, False, True],
                        })

print(df)
```

	nome	documento	idade	habilitado
0	Fulano	5342	25	False
1	Beltrano	1876	15	False
2	Sicrano	1872	43	True

Neste exemplo, `'nome'`, `'documento'`, `'idade'` e `'habilitado'` são os rótulos das colunas, que podemos utilizar para acessar, filtrar ou manipular os dados armazenados no `DataFrame`.

Por exemplo, se quisermos acessar toda a coluna `'idade'` para calcular a idade média dos usuários cadastrados, podemos fazê-lo com:

```
idade_media = df['idade'].mean()
print(f'idade média: {idade_media:.1f} anos')
```

```
idade média: 27.7 anos
```

Ou ainda, se desejarmos filtrar apenas os usuários não habilitados:



```
nao_habilitados = df[df['habilitado'] == False]
print(nao_habilitados)
```

	nome	documento	idade	habilitado
0	Fulano	5342	25	False
1	Beltrano	1876	15	False

Ou apenas os maiores de idade:

```
maiores_idade = df[df['idade'] >= 18]
print(maiores_idade)
```

	nome	documento	idade	habilitado
0	Fulano	5342	25	False
2	Sicrano	1872	43	True

Ou se desejarmos, não filtrar, mas ordenar pela idade:

```
df_ordenado = df.sort_values(by='idade')
print(df_ordenado)
```

	nome	documento	idade	habilitado
1	Beltrano	1876	15	False
0	Fulano	5342	25	False
2	Sicrano	1872	43	True

As possibilidades são inúmeras. A biblioteca provê meios de acessar, filtrar, manipular e transformar dados de forma expressiva e eficiente — tudo com poucas linhas de código.

Note ainda que, neste exemplo introdutório, definimos explicitamente quatro colunas ('nome', 'documento', 'idade' e 'habilitado'), mas, ao criarmos o `DataFrame`, surgiu uma coluna adicional (a primeira), que indica o número da linha. Essa coluna, chamada `index` no `pandas`, possui um papel especial: ela permite indexar não apenas as colunas, mas também as linhas desejadas.

Veja o exemplo:

```
selecionado = df.loc[1] # lê linha1
print(selecionado)
```

	nome	documento	idade	habilitado
1	Beltrano	1876	15	False

Name: 1, dtype: object

Ou ainda, podemos modificar um campo específico:

```
df.loc[1, 'nome'] = 'Beltrano Jr.' # escreve linha1, coluna 'nome'
df.loc[1, 'idade'] = 16             # escreve linha1, coluna 'idade'
print(df)
print(selecionado)
```

	nome	documento	idade	habilitado
0	Fulano	5342	25	False
1	Beltrano Jr.	1876	16	False
2	Sicrano	1872	43	True

Por padrão, quando não especificado, o `pandas` utiliza como `index` um inteiro sequencial. No entanto, é possível personalizá-lo com qualquer valor único, como strings, datas, timestamps ou qualquer identificador que faça sentido para a aplicação.

No nosso exemplo, um bom candidato a `index` seria 'documento', já que seus valores são únicos para cada usuário e representam uma forma mais precisa de identificar um indivíduo do que a simples ordem de cadastro (`index` padrão).



Para isso o código pode ser modificado como no exemplo abaixo:

```
import pandas as pd

df = pd.DataFrame(data={'nome': ['Fulano', 'Beltrano', 'Sicrano'],
                        'documento': [5342, 1876, 1872],
                        'idade': [25, 15, 43],
                        'habilitado': [False, False, True],
                        })

df = df.set_index('documento') # seta a coluna 'documento' como index

print(df)
print('-'*38, '\nUsuário 1872:')
print(df.loc[1872]) # agora podemos acessar uma linha pelo num. do documento
```

documento	nome	idade	habilitado
5342	Fulano	25	False
1876	Beltrano	15	False
1872	Sicrano	43	True

Usuário 1872:

nome	Sicrano
idade	43
habilitado	True

Name: 1872, dtype: object

Por padrão, métodos como `df.set_index()`, assim como a maioria dos métodos de `DataFrame`, não modificam a instância original (*inplace*), mas retornam uma nova cópia modificada. Essa escolha de design do `pandas` visa evitar efeitos colaterais indesejados, o que é especialmente valioso em análises de dados complexas.¹

Um recurso especialmente relevante do `pandas` é sua capacidade de importar e exportar dados de forma simples e eficiente.

No link https://github.com/j-Lago/pythonTutorial/blob/master/data/back_emf.csv, encontra-se um arquivo no formato `.csv` (*Comma-Separated Values*) contendo uma série temporal da força contraeletromotriz induzida de um motor síncrono de ímãs permanentes, registrada por meio de um osciloscópio. O arquivo possui duas colunas: `'time'` e `'voltage'`, como ilustrado a seguir:

```
back_emf.csv

time,      voltage
-0.010000046, 205.108
-0.0099998456, 207.107
-0.0099996456, 205.908
      :           :
0.010000154,  519.367
```

Podemos importar essa tabela para um `DataFrame` com:

```
import pandas as pd

df = df = pd.read_csv('back_emf.csv', index_col='time')

print(df.head()) # imprime as linhas iniciais do dataframe
```

time	voltage
-0.010000	205.108
-0.010000	207.107

¹Caso se deseje modificar o objeto diretamente, o que pode ser vantajoso em estruturas volumosas e transformações simples, é possível utilizar o argumento `inplace=True`, suportado pela maioria dos métodos da biblioteca.



```
-0.010000  205.908  
-0.009999  203.909  
-0.009999  204.308
```

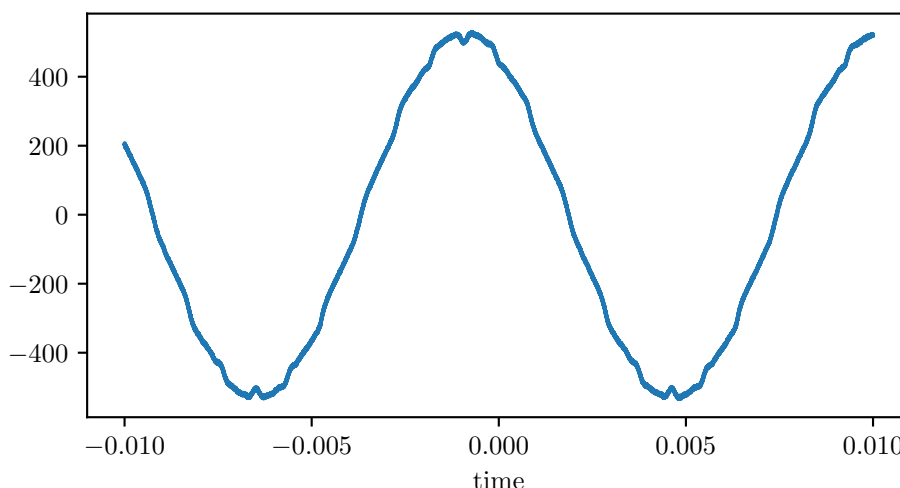
Uma vez importado, os dados podem ser analisados e modificados utilizando todos os recursos oferecidos pelo `pandas`. Essa biblioteca também se integra de forma eficiente com `numpy` e `matplotlib`, permitindo visualizações e operações numéricas avançadas. É possível, por exemplo, gerar `arrays` a partir dos dados de uma coluna específica do `DataFrame`:

```
voltage = df.to_numpy()  
print(type(voltage))  
print(voltage)
```

```
<class 'numpy.ndarray'>  
[[205.108]  
 [207.107]  
 [205.908]  
 ...  
 [521.366]  
 [522.566]  
 [519.367]]
```

Ou gerar um gráfico com os valores:

```
df['voltage'].plot()  
plt.show()
```



Por fim, o `pandas` também permite a exportação de dados por meio de diversos métodos, como `df.to_csv()`, `df.to_excel()` e `df.to_json()`, entre outros.

Essa seção apenas introduz a biblioteca `pandas`, que oferece uma ampla variedade de métodos para manipulação e análise de dados tabulares. Informações adicionais, bem como tutoriais e referências detalhadas, estão disponíveis na documentação oficial: <https://pandas.pydata.org/docs/>.

Capítulo 7

Exercícios

Diferente da lista de exercícios anterior, cujo objetivo era desenvolver o raciocínio lógico e familiarizar o leitor com os mecanismos básicos da linguagem Python, esta nova lista propõe desafios mais complexos, voltados para situações comuns no cotidiano profissional de engenheiros.

O foco agora é estimular a capacidade de resolver problemas e de traduzi-los para o ambiente computacional, utilizando o Python como ferramenta de apoio.

Questão 7.1: `pvcell`

Uma célula fotovoltaica é basicamente uma junção P–N exposta à luz, o que possibilita conversão da energia luminosa em energia elétrica.

Como essa junção funciona, essencialmente, como um diodo de silício, a tensão gerada é muito baixa, e a relação entre tensão e corrente apresenta um comportamento fortemente não linear.

A Figura 7.1 ilustra um modelo equivalente da célula na forma de um circuito eletrônico. Nesse modelo, a fonte de corrente e o diodo representam a dependência da célula em relação à irradiância e à temperatura.

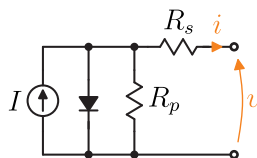


Figura 7.1: Modelo de uma célula fotovoltaica.

O circuito da Figura 7.1 pode ser equacionado por (7.1), onde I , S e T dependem das características construtivas da célula além da irradiância e temperatura. Ou seja, a rigor, esses parâmetros variam com as condições de operação às quais a célula é submetida. Neste exercício, para fins de simplicidade, considere-os constantes.

$$i = I - S \left(e^{\frac{v - iR_s}{T}} - 1 \right) - \frac{v - iR_s}{R_p} \quad (7.1)$$



A resolução da (7.1) permite calcular a corrente, a tensão e, por consequência, a potência gerada para uma determinada condição de irradiância e temperatura. A principal dificuldade nessa resolução está na interdependência do termo exponencial com a tensão e a corrente da célula, o que torna a equação transcendental, impedindo o isolamento direto de i . Dessa forma, a solução só pode ser obtida por meio de métodos numéricos, utilizando aproximações sucessivas.

Diversos métodos numéricos podem ser utilizados para resolver a (7.1). Em particular, o método de Newton permite obter uma solução aproximada por meio de iterações sucessivas, atualizando a estimativa a cada passo. Se o método convergir, essa nova estimativa tende a se aproximar progressivamente da solução exata. Aplicando o método de Newton à (7.1), obtém-se:

$$i_n = i_{n-1} - \delta_{n-1} \tag{7.2}$$

sendo

$$\delta_n = \frac{I - S \cdot \exp\left(\frac{v - R_s \cdot i_n}{T} - 1\right) - \frac{v - R_s \cdot i_n}{R_p} - i_n}{-1 - \frac{S \cdot R_s}{T} \cdot \exp\left(\frac{v - R_s \cdot i_n}{T}\right) - \frac{R_s}{R_p}} \tag{7.3}$$

A estimativa da corrente, dada por (7.2), deve ser atualizada iterativamente para $n = 1, 2, \dots$, até que o valor absoluto de δ_n seja inferior à tolerância estabelecida. Em outras palavras, as equações (7.2) e (7.3) devem ser aplicadas sucessivamente enquanto $|\delta_n| > \epsilon$.

Como se trata de um método iterativo, cada nova estimativa da corrente depende da anterior. Assim, o cálculo de i_1 requer o conhecimento prévio de i_0 . Para iniciar o algoritmo, assume-se $i_0 = 0$.

Etapa 1

Escreva uma função `i_cell(v: float) -> float` que calcule e retorne a corrente da célula para uma dada tensão passada como argumento.

Utilize as seguintes constantes:

símbolo	valor	U.M.
R_s	0.002	Ω
R_p	10.0	Ω
I	8.0	A
S	0.02	A
T	0.085	V
ϵ	0.001	A

Um teste inicial da função gerada pode ser realizado com:

```
def i_cell(v: float) -> float:
    # seu código aqui

print(f"i_cell(0.4) = ")
print(f"i_cell(0.5) = ")

i_cell(0.4) = 7.275763496758507
i_cell(0.5) = 5.64008091541083
```



Etapa 2

Uma vez realizada essa validação inicial, escreva um script que trace uma curva da corrente em função da tensão nos terminais da célula tensão. O script deve gerar uma figura semelhante a da Figura 7.2.

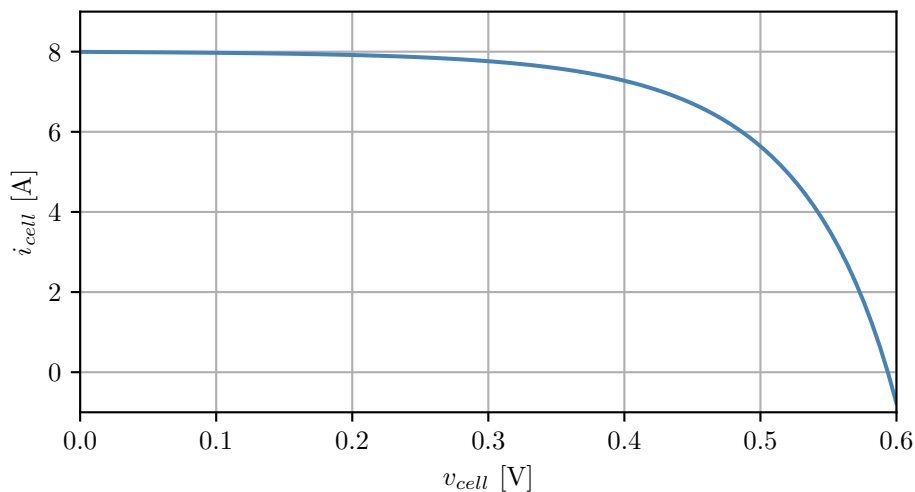


Figura 7.2: Resultado gráfico esperado do script para teste de `i_cell`.

Etapa 3

Utilize algum método do pacote `scipy.optimize` obter o ponto de máxima potência, e represente-o no gráfico de potência por tensão da célula. O script deve gerar uma figura semelhante a da Figura 7.3.

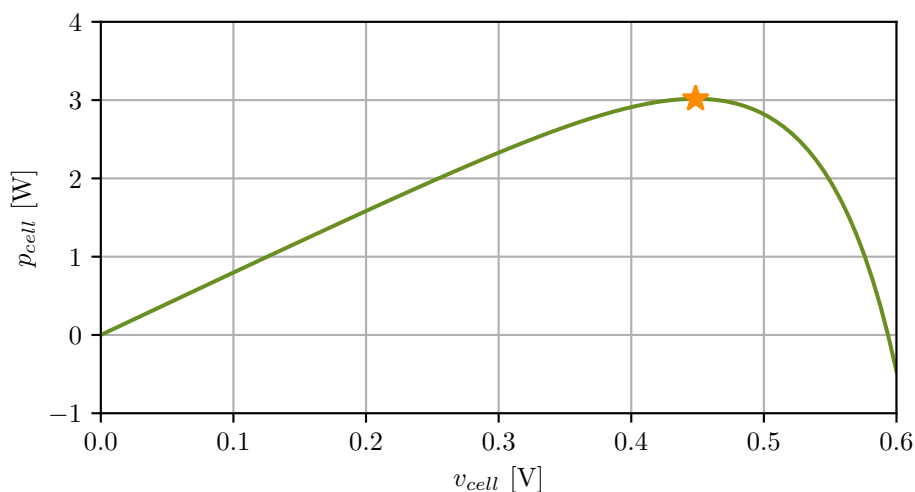


Figura 7.3: Resultado gráfico esperado do script de MPP.



Questão 7.2: pll

Em sistemas de geração fotovoltaica, a energia solar captada pelos painéis é convertida em energia elétrica em corrente contínua. Como a maioria desses sistemas não possui meios de armazenamento, e como energia se conserva, toda a energia gerada pelos painéis deve ser imediatamente injetada na rede elétrica.

Para escoar essa produção, os inversores devem não apenas executar a conversão CC-CA, mas fazê-la de forma a controlar a injeção de potência na rede, garantindo o equilíbrio energético e consequentemente a estabilidade do barramento CC.

Para isso, a tensão imposta pelo inversor deve estar em sincronismo com a da rede. O controle da potência ativa depende diretamente da defasagem entre a tensão da rede e a tensão imposta pelo inversor. Além disso, como a rede elétrica opera com frequência variável em torno dos 60 Hz, o inversor deve estimar continuamente a fase da tensão da rede a fim de realizar os ajustes necessários para manter o sincronismo e o ângulo de carga desejado.

Essa estimação não é trivial já que, além das variações da tensão e frequência em regime permanente, a rede ainda apresenta efeitos transitórios impulsivos e oscilatórios, distorções harmônicas, flutuações, ruídos, afundamentos, etc. Isso é ainda mais desafiador em sistemas monofásicos, onde amplitude, frequência e fase devem ser obtidas a partir de medições de um único sinal de tensão, sujeito aos fenômenos já citados.

Na prática, existem algumas estratégias para se obter uma estimativa instantânea e precisa da fase da tensão da rede, sendo que a maioria delas é baseada na estrutura SOGI-PLL, cujo diagrama de blocos básico é apresentado na Figura 7.4.

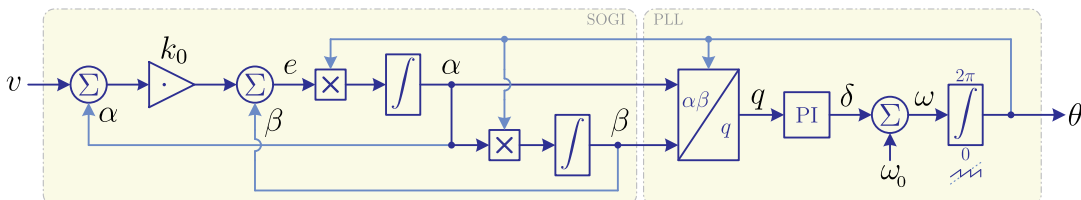


Figura 7.4: SOGI-PLL.

O estimador da Figura 7.4 pode ser discretizado e reescrito na forma de equações a diferenças, apresentadas nas equações (7.4–7.11), possibilitando sua implementação em controladores digitais embarcados.

O algoritmo discretizado recebe um sinal amostrado da tensão de entrada, tipicamente com frequência de amostragem na ordem de algumas dezenas de kHz, dependendo da frequência de comutação utilizada no inversor.

Com base nessas amostras, estima-se a fase da tensão quase instantânea (com atraso de uma amostra). O sinal de fase estimado é utilizado nas malhas de controle das correntes e potências ativa e reativa injetadas na rede (não abordados nessa questão).

$$e_n = k_0 (v_n - \alpha_{n-1}) - \beta_{n-1} \quad (7.4)$$

$$\alpha_n = w_{n-1} k_1 (e_n + e_{n-1}) + \alpha_{n-1} \quad (7.5)$$

$$\beta_n = w_{n-1} k_1 (\alpha_n + \alpha_{n-1}) + \beta_{n-1} \quad (7.6)$$



$$d_n = \alpha_n \sin(\theta_{n-1}) - \beta_n \cos(\theta_{n-1}) \quad (7.7)$$

$$q_n = \alpha_n \cos(\theta_{n-1}) + \beta_n \sin(\theta_{n-1}) \quad (7.8)$$

$$\delta_n = k_2 q_n + k_3 q_{n-1} + \delta_{n-1} \quad (7.9)$$

$$\omega_n = \omega_0 + \delta_n \quad (7.10)$$

$$\theta_n = k_1 (\omega_n + \omega_{n-1}) + \theta_{n-1} \pmod{2\pi} \quad (7.11)$$

A estimativa de fase da rede (*theta*) obtida ainda deve ser validada. O `pll` pode demorar alguns ciclos para se sincronizar com a rede (ou atracar).

Etapa 1

Em <https://github.com/j-Lago/pythonTutorial/blob/master/data/> você encontrará um arquivo `grid_voltage.csv` com a tensão (**coluna 2**) amostrada de alguns ciclos da rede elétrica e um registro temporal (**coluna 1**) de cada amostra.

Escreva um script que importe esse sinal para um `array`, e para cada amostra (cada elemento do `array`) execute os cálculos (7.4–7.11). Guarde os resultados de cada amostra de v (tensão de entrada), α (tensão filtrada), d (componente de eixo direto da tensão em base síncrona), q (componente em quadratura) e θ (fase estimada pelo PLL).

Assuma ainda, de forma simplificada, que o sincronismo foi obtido no instante do primeiro cruzamento por zero do sinal θ (saída do PLL) em que $\arctan(q/d) < 0.03 \text{ rad} (\approx 2^\circ)$ e $d > 250 \text{ V}$. Identifique esse instante bem como os demais cruzamentos por zero subsequentes.

Utilize as seguintes constantes para a implementação do controlador e das integrações:

$$\begin{aligned} k_0 &= 0.7 \\ k_1 &= 2.84 \cdot 10^{-6} \\ k_2 &= 0.385715277950311 \\ k_3 &= -0.385713293478261 \\ \omega_0 &= 376.9911184307752 \end{aligned} \quad (7.12)$$

Gere e exporte uma figura apresentando a evolução temporal de cada uma desses sinais, bem como alguma indicação do instante que o PLL conseguiu se sincronizar (atracar) com a rede elétrica. A Figura 7.5 mostra um exemplo de como os resultados do script podem ser apresentados.

Etapa 2

Exporte para um arquivo `.csv` a porção do sinal de tensão de entrada compreendida entre a detecção inicial da sincronização até a amostra que antecede o último cruzamento por zero do ângulo θ , ou seja, o período que compreende todos os ciclos inteiros em que o PLL estava sincronizado. Exporte além da tensão, o período correspondente do vetor de tempo e do ângulo de saída do PLL.

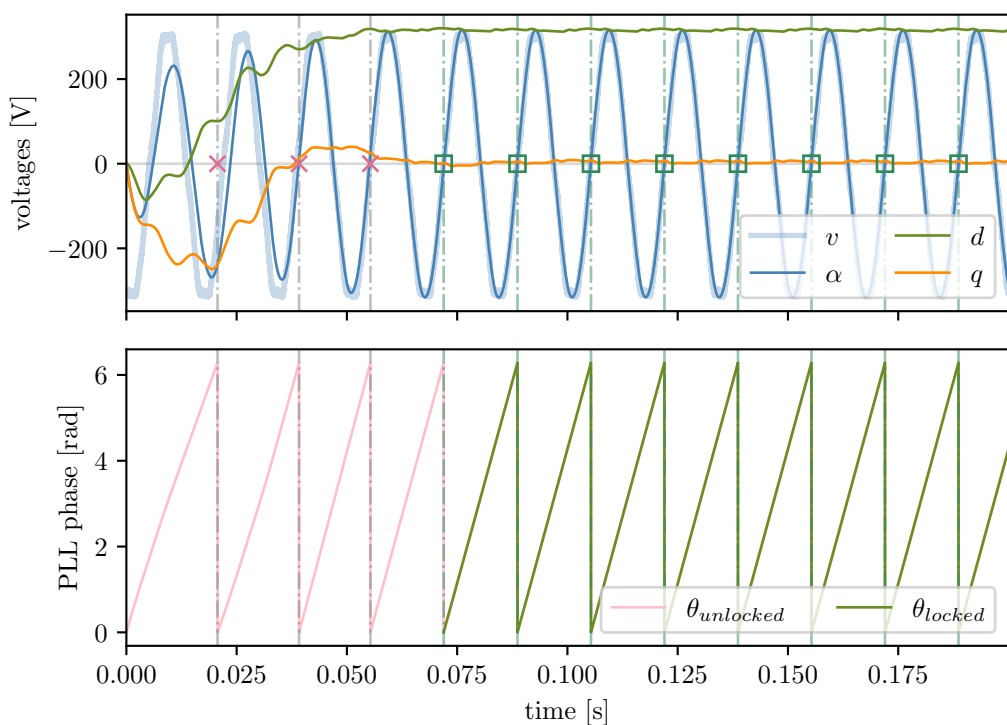


Figura 7.5: Resultado esperado do script.

amostras_sincronizadas.csv		
time,	phase,	voltage
0.07194288000000001,	0.0012548603076290021,	1.4167228992250036
0.07194856,	0.0033896994694907535,	5.138057712393213
0.07195424,	0.005524549922962972,	6.812246200963261
0.07195992000000001,	0.007659409882508262,	0.30519810343556353
0.07196559999999999,	0.009794267899159199,	0.4020358458949955
:	:	:
0.1886612,	6.280879889823333,	8.224179145671663

Etapas 3

Os dados exportados na etapa anterior consistem num intervalo da tensão da rede com ciclos inteiros. Isso nos permite calcular o valor rms da tensão amostrada da rede através de (7.13):

$$V_{\text{rms}} = \sqrt{\frac{1}{N} \sum_{n=0}^{N-1} v_n^2} \tag{7.13}$$

Escreva uma função `rms(path: str) -> tuple[float, float]` que recebe como argumento um texto indicando o caminho do arquivo exportado na questão anterior e que calcule e retorne o valor rms da tensão e sua frequência fundamental média.

```
def rms(path: str) -> tuple[float, float]:
    # seu código aqui

vrms, f1 = rms('amostras_sincronizadas.csv')
print(f"A amostra de tensão possui valor rms de {vrms:.2f} V e frequência {f1:.2f} Hz")
```

Saída esperada:



A amostra de tensão possui valor rms de 223.03 V e frequência 59.98 Hz

[†]Note que (7.13) só é válida para um vetor $\mathbf{x}$ contendo ciclos inteiros da componente fundamental. Ainda, para melhorar a precisão do método, recomenda-se realizar o cálculo considerando ao menos dez ciclos completos da fundamental (o que não é possível nesse exemplo pelo tamanho da amostra inicial).

Questão 7.3: consumo_horario

No repositório <https://github.com/j-Lago/pythonTutorial/blob/master/data/> você encontrará o arquivo `subestacao.csv`, que contém medições de diversas grandezas elétricas registradas por um analisador de potência instalado na subestação de uma empresa da região.

Os dados foram registrados em intervalos de 10 minutos, representando métricas estatísticas (como média, valor máximo e mínimo) calculadas para cada janela de amostragem, ao longo de um período de duas semanas.

Importe os dados para um `pandas.DataFrame`, realize uma análise que gere um gráfico com o consumo médio por hora do dia, destacando o horário que, em média, tem o maior consumo.

Considere apenas os dias úteis, desconsiderando os finais de semana.

A Figura 7.6 mostra um exemplo de como os resultados do script podem ser apresentados.

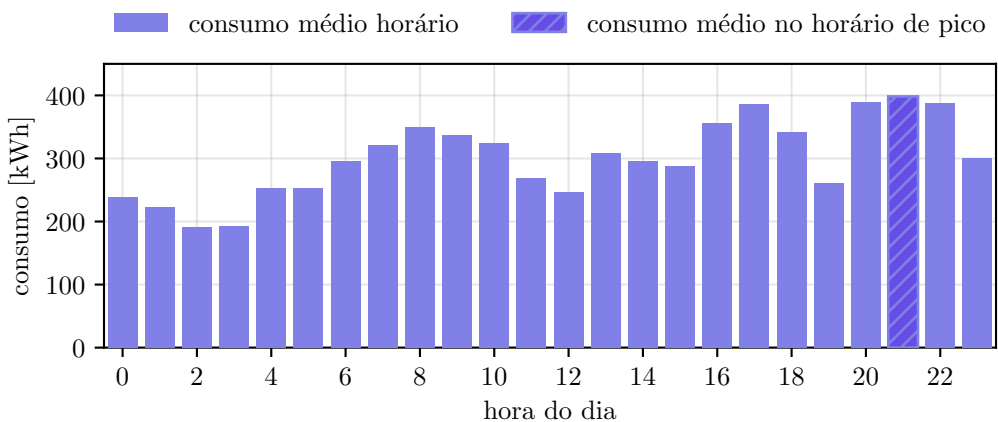


Figura 7.6: Resultado esperado.

Dica:

A primeira coluna do arquivo representa um *timestamp* das amostras e, por isso, é recomendável utilizá-la como `index` do `DataFrame`. Ao converter esses valores de *timestamp* para o tipo `datetime` (que, por padrão, seriam importados como `str`), você passa a contar com métodos bastante úteis da API de datas e horas do `pandas`, como `.hour`, `.weekday`, entre outros — o que pode facilitar significativamente a análise temporal requerida neste exercício.

```
df = pd.read_csv('subestacao.csv',
                 index_col='time', parse_dates=['time'], date_format='%Y-%m-%d %H:%M:%S')
```



Questão 7.4: `fft`

A transformada discreta de Fourier (DFT), bem como sua versão otimizada, a transformada rápida de Fourier (FFT), possui diversas aplicações, entre elas o processamento e a filtragem de sinais temporais discretizados.

Vamos explorar essa técnica para atenuar eventuais ruídos de medição presentes em um sinal de tensão amostrado, realçando suas componentes harmônicas relevantes.

Etapa 1

Desenvolva um script que sintetize a evolução temporal da tensão em uma fase da rede de baixa tensão. A discretização do sinal deve conter 1024¹ amostras uniformemente no intervalo de tempo entre 0 e 50ms. A forma de onda resultante deve incluir uma componente fundamental com valor eficaz de 220V e frequência de 60Hz, além de uma componente harmônica de 5ª ordem com valor eficaz de 40V e fase de 128.4°. Adicione também um ruído aleatório com amplitude de 300V e média nula (consulte a `np.random.rand`).

Etapa 2

Utilize as funções do módulo `numpy.fft` para calcular a transformada de Fourier do sinal gerado na etapa anterior, determinando a amplitude de cada componente de frequência presente em seu espectro.

Em seguida, filtre o espectro resultante, zerando as componentes cuja magnitude seja inferior a $10 \cdot 1024$.

Aplice a transformada inversa de Fourier (`ifft`) para reconstruir o sinal de tensão já filtrado.

Por fim, gere um gráfico comparativo que sobreponha o sinal filtrado ao sinal original, de forma a destacar visualmente os efeitos do processo de filtragem, conforme exemplificado na Figura 7.7.

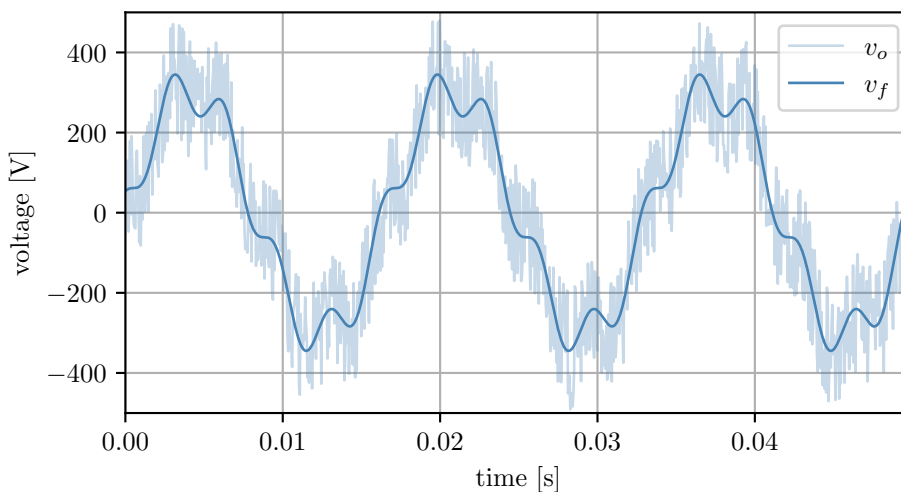


Figura 7.7: Resultado esperado.

¹Para que essa técnica seja empregada de forma eficiente, é necessário que o número de amostras do sinal corresponda a uma potência de dois.



Questão 7.5: **inverter**

A evolução dos inversores comutados (conversores CC-CA) foi fundamental para viabilizar o avanço das tecnologias de geração eólica e solar fotovoltaica. Além da própria evolução tecnológica empregada na fabricação dos dispositivos semicondutores, foi a imposição de uma limitação para operar os transistores apenas em dois estados, corte e na saturação, que possibilitou seu uso no processamento de potência de forma eficiente.

Idealmente, nesses dois estados, o transistor não apresentaria perdas. No entanto, essa limitação, embora essencial para lidar com altos níveis de potência, impede que os inversores consigam impor em sua saída níveis de tensão contínuos, como tensões senoidais. Essa forma seria desejável para injetar potência na rede (também senoidal).

Um inversor convencional de tensão imposta monofásico geralmente é composto por 4 transistores, cada um operando como uma chave eletrônica aberta ou fechada. Excluindo algumas combinações que levariam a curto-circuitos e à não imposição da tensão de saída, restam as 4 permutações válidas dos estados desses interruptores que resultam em apenas 3 níveis possíveis de tensão de saída: positivo ($+V_{dc}$), negativo ($-V_{dc}$) e zero (0).

A solução adotada para contornar essa limitação consiste em emular valores intermediários de tensão por meio da comutação entre esses três níveis válidos. Controlando-se o tempo de permanência de cada estado ativo, é possível ajustar a tensão média quase instantânea na saída do conversor.

Ao variar a relação entre os tempos de condução e bloqueio seguindo uma referência senoidal, obtém-se uma forma de onda comutada que, após filtragem adequada, se aproxima de uma tensão senoidal. Esse processo é conhecido como modulação.

Além de permitir a síntese de formas de onda que aproximam tensões senoidais, através da modulação apropriada podemos impor além do formato, a amplitude, frequência e fase da tensão de saída, o que é essenciais para o controle da energia injetada na rede elétrica.

A Figura 7.8 apresenta um circuito equivalente simplificado de um inversor monofásico conectado à rede elétrica convencional, e poderia modelar a interface com a rede em um sistema de geração fotovoltaica. Esse circuito está dividido em três partes: inversor, que através da lógica de modulação entrega tensão comutada à saída; um par RL série representando a impedância do filtro de saída do inversor combinado com a impedância da rede; e uma fonte de tensão, representando a rede de energia como um barramento infinito.

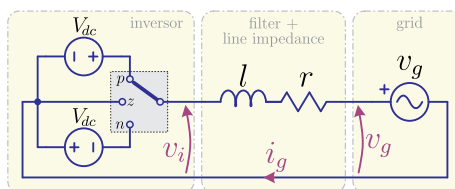


Figura 7.8: Modelo de um inversor monofásico conectado à rede elétrica.

A tensão do inversor (v_i) é comutada e possui apenas três níveis: positiva no valor de V_{dc} , negativa no valor de $-V_{dc}$ ou zero. Quem define o estado instantâneo da tensão do inversor é o modulador, e o faz baseado na comparação de dois sinais intermediários denominados moduladora (v_m) e portadora (v_c). A moduladora representa o sinal que desejamos ter na saída do inversor enquanto a portadora carrega a informação da frequência de comutação.



Essa técnica é conhecida como modulação por largura de pulso (PWM) senoidal, e é uma das modalidades de modulação mais simples e empregada em inversores, embora não seja a única.

A moduladora é o sinal que desejamos emular, e pode ser definida por:

$$v_m(t) = \sqrt{2} \cdot V_i \cdot \sin(\theta + \varphi_i) \quad (7.14)$$

onde V_i é o valor rms que desejamos impor na saída do inversor, θ a fase instantânea da rede elétrica e φ_i a fase da tensão imposta pelo inversor relativa à rede elétrica.

A portadora utilizada é um sinal triangular definido por:

$$v_c(t) = V_{dc} \cdot \left(4 \cdot \text{abs}(\text{mod}(f_c \cdot t, 1) - \tfrac{1}{2}) - 1\right) \quad (7.15)$$

onde t é o tempo, V_{dc} é a tensão do barramento CC e f_c a frequência da portadora, que definirá a frequência de comutação dos transistores e, por consequência, a da tensão comutada na saída do inversor.

Finalmente, a tensão do inversor é obtida através de comparações entre a moduladora e a portadora, e é dada por:

$$v_i(t) = V_{dc} \cdot [s_p(t) - s_n(t)] \quad (7.16)$$

sendo:

$$s_p(t) = \begin{cases} 1 & , v_m(t) \geq v_c(t) \\ 0 & , \text{outros casos} \end{cases} \quad (7.17)$$

e

$$s_n(t) = \begin{cases} 1 & , v_m(t) \leq -v_c(t) \\ 0 & , \text{outros casos} \end{cases} \quad (7.18)$$

A função de transferência da impedância do filtro e rede pode ser aproximada por uma equação a diferenças, por:

$$i_{g_n} = k_0 \cdot v_{i_n} - k_0 \cdot v_{g_n} + k_1 \cdot i_{g_{n-1}} \quad (7.19)$$

onde k_0 e k_1 são constantes definidas pela indutância, resistência e frequência de amostragem dos sinais, enquanto o índice n representa a n -ésima amostra dos sinais das tensões e corrente.

Implemente uma função que simule as correntes resultantes em um sistema fotovoltaico conectado à rede. Sua assinatura pode ser:

```
def inversor(df: pd.DataFrame, Vi: float, phii: float) -> pd.DataFrame:
    # seu código aqui
    return df
```

A função recebe como primeiro argumento um `DataFrame` contendo as séries temporais `'time'`, `'phase'` e `'voltage'`, que representam, respectivamente, o tempo (t), a fase instantânea da tensão da rede (θ) e a própria tensão da rede elétrica (v_g).

Além disso, a função recebe mais dois parâmetros numéricos: `Vi`, correspondente ao valor eficaz da tensão imposta pelo inversor (V_i), e `phii`, que define a defasagem da tensão de saída do inversor em relação à fase da rede elétrica (φ_i).

A função deve resolver as equações de modulação e simulação (eq. a diferenças) e retornar um `DataFrame` com as séries: t , θ , v_g , v_m , v_c , v_i , i_g .



Escreva ainda um scrip que gere um gráfico com esses sinais.

Para o argumento `df`, utilize os dados da porção sincronizada das séries de tempo, fase e tensão que foram geradas e exportadas para um arquivo `.csv` na Etapa 3 da questão `p11`.

Utilize os seguintes valores para os parâmetros adicionais da função: `Vi=250` e `phii=0.4`.

Assuma as seguintes constantes, previamente calculadas para a frequência de amostragem do problema:

$$\begin{aligned} V_{dc} &= 400 \text{ V} \\ f_c &= 600 \text{ Hz} \\ k_0 &= 3.3333 \cdot 10^{-4} \\ k_1 &= 0.99963 \end{aligned} \tag{7.20}$$

A Figura 7.9 apresenta o resultado esperado:

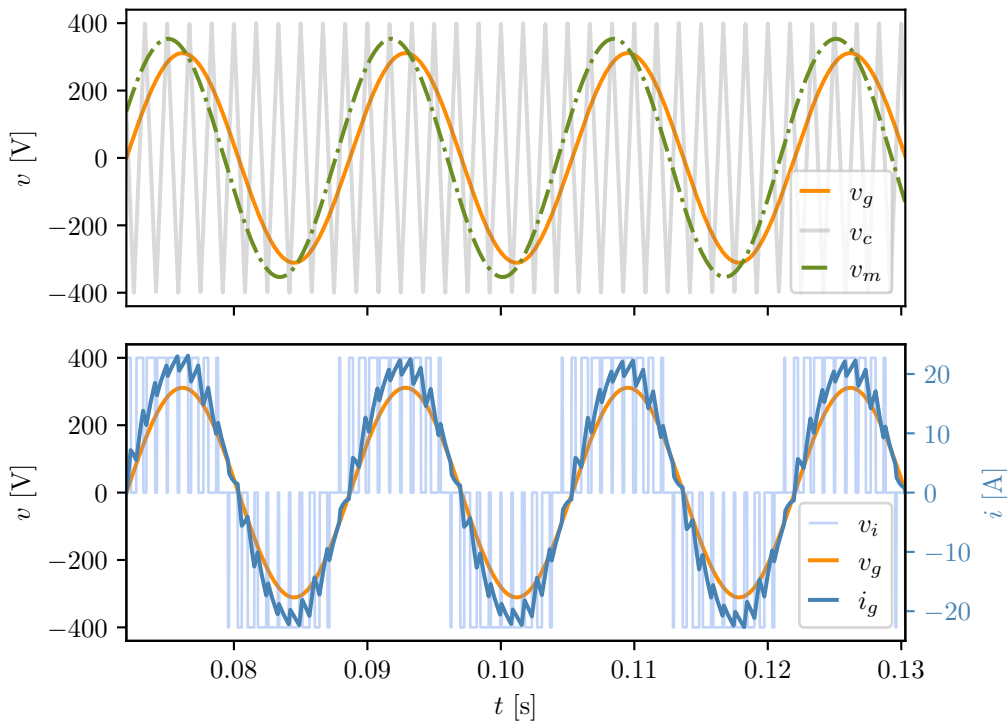


Figura 7.9: Resultado gráfico esperado do script.

Referências

- [1] JetBrains. *PyCharm Guide*. <https://www.jetbrains.com/pycharm/guide/>. Documentação oficial do PyCharm. 2024.
- [2] Mark Lutz. *Learning Python*. 5th. O'Reilly Media, 2013.
- [3] *Matplotlib Documentation*. <https://matplotlib.org/stable/contents.html>. Documentação oficial da biblioteca Matplotlib. 2024.
- [4] Eric Matthes. *Python Crash Course*. 2nd. No Starch Press, 2019.
- [5] *NumPy Documentation*. <https://numpy.org/doc>. Documentação oficial da biblioteca NumPy. 2024.
- [6] *pandas Documentation*. <https://pandas.pydata.org/docs/>. Documentação oficial da biblioteca pandas. 2024.
- [7] *Python in Visual Studio Code*. <https://code.visualstudio.com/docs/python/python-tutorial>. Documentação da extensão Python para VS Code. 2024.
- [8] *Python Language Reference*. <https://docs.python.org/3/reference/>. Referência da linguagem Python. 2024.
- [9] *SciPy Documentation*. <https://docs.scipy.org/doc/scipy/>. Documentação oficial da biblioteca SciPy. 2024.
- [10] Al Sweigart. *Automate the Boring Stuff with Python*. 2nd. No Starch Press, 2019.
- [11] *The Python Tutorial*. <https://docs.python.org/3/tutorial/>. Documentação oficial do Python. 2024.